

MATEMATICA

Produs scalar

Ec param a drepte: $P = P_1 + t(P_2 - P_1) = (1-t)P_1 + tP_2$.

$D_1 \perp D_2 \Leftrightarrow m_1 = (-1) \cdot m_2$; $\cos(\theta) = 0$; $u \cdot v = 0$ (vectorii dreptelor).

$u \cdot v = u_x v_x + u_y v_y + u_z v_z$. Proprietati: comunicativ, distributiv fata de adunare. Utilizari: lungimea unui vector [$\text{length}(v) = \|v\| = \sqrt{v \cdot v}$], distanta dintre 2 puncte [$v = Q - P$, $\text{distanța}(P, Q) = \text{length}(v)$], normalizarea unui vector: [$\hat{v} = v / \|v\|$], unghiul dintre 2 vectori [$u \cdot v = |u| |v| \cos(\theta)$, $u \cdot v < 0 \Rightarrow \theta > 90^\circ$; $u \cdot v = 0 \Rightarrow \theta = 90^\circ$; $u \cdot v > 0 \Rightarrow \theta < 90^\circ$], pozitia punctului fata de un plan, proiectia unui vector pe altul [u proiectat pe $v = \text{vector de lungime } u \cdot \hat{v}$], distanta de la un punct la un plan.

Produs vectorial

= un vector perpendicular pe ambii vectori

$v = v_1 \times v_2 = \text{determinant} : \begin{vmatrix} i & j & k \\ x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \end{vmatrix}$. Regula mainii drepte. Aplicatii: calculeaza vectorul normal la un poligon din plan [$n_1 = e_1 \times e_2$, e_1, e_2 laturi consecutive ale unui poligon din plan] ; conventie: normalele in afara unui obiect $\Rightarrow$ citire in sensul acelor de ceasornic ; normalele in interiorul obiectului $\Rightarrow$ citire in sens invers acelor de ceasornic].

Distanța pana la o linie

Ecuatia liniei: $P(t) = P_0 + tv$, t parametru, v vectorul directie ; Distanța de la R la linie = $P_0 + ((R - P_0)v / (v \cdot v))v$.

Relatia punct-punct

Ordinea punctelor P, Q, R e data de determinantul $[x_1, y_1, 1 | x_2, y_2, 1 | x_3, y_3, 1]$ astfel: $\text{sign}(D) = 1(+)$ $\Rightarrow$ sens invers acelor de ceas, $\text{sign}(D) = 0$ $\Rightarrow$ puncte coliniare, $\text{sign}(D) = -1(-)$ $\Rightarrow$ sensul acelor de ceas.

Relatia punct-linie

$P(x_0, y_0)$ este pe linie daca P verifica $P = P_1 + t(P_2 - P_1)$. Sistemul ar trebui sa aiba o solutie reala: $x_0 = x_1 + t(x_2 - x_1)$ si $y_0 = y_1 + t(y_2 - y_1)$, altfel P este in afara liniei. Discutie dupa t : $t = 0 \Rightarrow P = P_1(x_1, y_1)$, $t = 1 \Rightarrow P = P_2(x_2, y_2)$, $0 < t < 1 \Rightarrow P$ intre P_1 si P_2 .

Relatia linie-linie

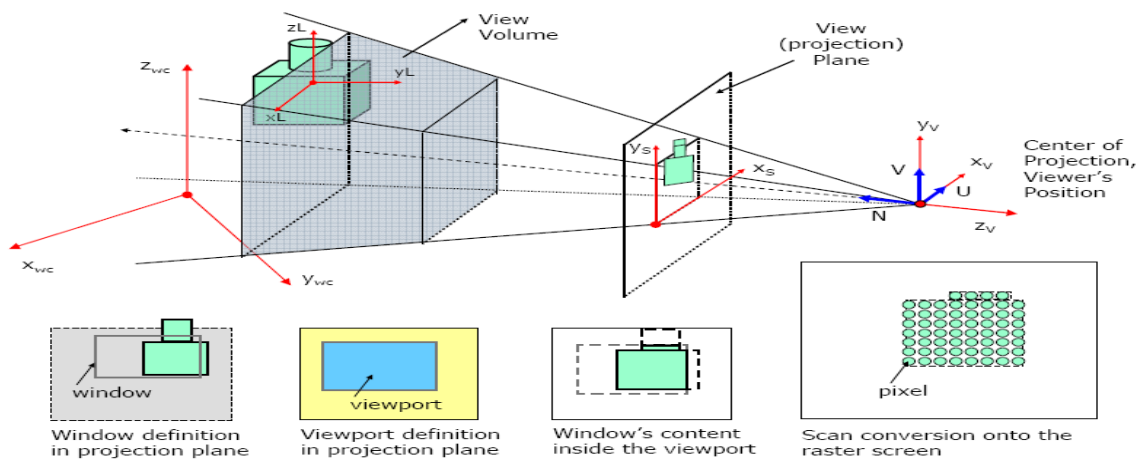
Sistemul a 2 linii: $P = (1-t)P_1 + tP_2$, $Q = (1-s)Q_1 + sQ_2$. Sistemul ar trebui sa aiba solutii reale in t si s . Discutie dupa t si s : $t < 0$ si $s < 0 \Rightarrow P$ inainte de P_1 si Q_1 , $0 < t < 1$ si $0 < s < 1 \Rightarrow P_1P_2$ intersecteaza Q_1Q_2 in P , $t > 1$ si $s > 1 \Rightarrow P$ dupa P_2 si Q_2 .

Relatia punct – obiect 3D convex

Se calculeaza ecuatia planului si functia: $F_i(x, y, z) = A_i x + B_i y + C_i z + D_i$ pt toate fetele obiectului Ω . Se calculeaza normalele pt a arata in afara obiectului 3D. $P(x_p, y_p, z_p)$ este localizat: daca cel puțin o functie $F(P) > 0 \Rightarrow P$ in afara Ω ; daca o functie $F_k(P) = 0 \Rightarrow P$ pe fata k ; daca $F_k(P) = F_q(P) = 0 \Rightarrow P$ pe muchia dintre fetele k si q ; daca r functii $F_{k_1}(P) = F_{k_2}(P) = \dots = F_{k_r}(P) = 0 \Rightarrow P$ pe dreapta dintre fetele $k_1, k_2, \dots, k_r$; daca toate functiile $F_k(P) > 0 \Rightarrow P$ in interiorul Ω ; altfel eroare.

SUPRAFETE ASCUNSE (Hidden line and surface removal algorithms)

Conceptual model of 3D viewing process



Graphical Processing Systems - UTCN

5

Viewing transformations pipeline: 1. 3D – coordonate ale lumii → Transforma in coordonate de vizualizare 3D – coordonate de vizualizare → Calculul umbrelor, decupare dupa volumul vizibil, eliminarea fetelor din spate 3D – coordonate de vizualizare → Proiectie pe planul de vizualizare, decupare dupa fereastra 2D – coordonate de vizualizare → Transformare in 2D 2D – coordonate normalizate → Rasterizare, Calcularea vizibilitatii, compozitia culorilor, texturi Coordonate dispozitivului fizic → Dispozitiv

For fiecare obiect din lume { det care parti ale obiectului nu sunt obstructionate de alte parti ale acestuia sau de alte obiecte. Deseneaza aceste parti in culoarea corespunzatoare }

Algoritmi in spatiul imagine: det ce este vizibil pt fiecare pizel in parte

For fiecare pixel din imagine { det care obiect care este proiectat pe pixel este mai aproape de observator. Deseneaza pixelul in culoarea corespunzatoare }

Sorting efficiency – coherence:

a) Obiect – daca un obiect este in intregime separat de altul comparatiile sunt necesare numai intre cele 2 obiecte, si nu intre fetele si muchiile componentelor

b) Fata – proprietatile suprafetelor variaza de obicei putin dealungul unei fete, permitand calculele facute pt o parte a unei fete sa fie modificate incremental si aplicate partiilor adiacente

c) Muchie – poate sa se modifice vizibil numai unde trece pe sub o alta muchie vizibila sau cand penetreaza o fata vizibila

d) Scanline – setul spatiilor vizibile ale obiectelor pt un scanline a unei imagini difera putin in mod normal de setul liniei precedente

e) Adancime – partiile adiacente ale aceleiasi suprafete sunt de obicei apropiate ca adancime. Odata ce adancimea unui punct de pe suprafata este calculata, adancimea celorlalte puncte de pe suprafata pot fi des determinate printr-o simpla ecuatie diferentiala

f) Cadru – imaginile a doua puncte succesive in timp sunt probabil f similare in ciuda micilor schimbari ale obiectului sau a punctelor de vedere

Floating Horizon Algorithm – Avem o suprafata exprimata prin $F(x,y,z)=0$ reprezentata prin curbe. Convertim problema din 3D in 2D prin intersectia suprafetei cu o serie de plane paralele pe axa Oz. Functia F este redusa la o curba in fiecare plan $y=f(x,z)$, unde z este constant.

for $z=z_1$ to z_n {for $x=x_{min}$ to x_{max} {render $y=f(x,z)$ }}

Back-face culling Algorithm – Se taie poligoanele care au normala indreptata opus fata de observator. Se foloseste produsul scalar. Daca $N \cdot V > 0$ atunci $\theta < 90$, inseamna ca normala e orientata in sens opus, deci poligonul se taie. Functioneaza doar pe poligoane convexe. Imbunatateste performanta de 2 ori.

Area Subdivision Algorithms.

WARNOCK ALG.: Lucram in spatiul imagine. Utilizam arii: daca putem spune cu certitudine ce poligon este vizibil atunci il afisam. Altfel, impartim aria in arii mai mici. Un poligon poate fi: continut in arie, intersectat cu aria, disjunct de arie, sa cuprinda aria. Cele disjuncte se elimina. Cele care intersecteaza se divid in disjuncte si continute. Daca sunt mai multe poligoane intr-o arie il afisam pe cel mai din fata. Daca nu putem determina ce sa afisam divizam din nou aria. Putem diviza pana la rezolutia maxima. Daca este disjunct se afiseaza fundalul, daca este continut sau intersectat se afiseaza fundalul si pe urma culoarea poligonului. Daca poligonul acopera aria si nu sunt poligoane disjuncte sau intersectate afisam culoarea poligonului inconjurator. EX: Impart suprafata in 4 patrute. Daca sunt patrute goale le notez cu 1; daca sunt patrute colorate in totalitate de un singur poligon le notez cu 3; daca sunt patrute continute in totalitate in ambele poligoane le notez cu 4; daca sunt patrute in care apare si spatiu gol si un poligon le notez cu 2; daca fondul, culoarea A sau culoarea B nu se suprapun impart patrutul in alte 4 patrute. Daca sunt mai multe poligoane intersectate unul din ele este in fata celorlalte. Pt a determina ordinea in care se vad poligoanele: P e obs, planul A al poligonului A si poligonul B; inlocuiesc toate varfurile lui B in ecuatia planului A si punctul P. Daca semnele sunt contrare atunci B e in spatele lui A, altfel e in fata.

WEILER-ATHERTON ALG.: Divide ecranul dupa laturile poligoanelor, nu dupa laturile dreptunghiurilor. Foloseste algoritmul lui Weiler de decupare a poligoanelor. Se sorteaza poligoanele dupa valoarea lui z. Cel mai apropiat poligon e folosit pt a decupa toate poligoanele. Rezulta 2 tipuri de poligoane: in interiorul si in exteriorul poligonului de decupare. Toate poligoanele din interiorul poligonului de decupare care sunt in spatele acestuia sunt sterse. Daca vreun poligon din lista celor interioare este mai apropiat decat poligonul de decupare atunci s-a facut o eroare la sortare. Fiecare astfel de poligon devine noul poligon de decupare si se continua recursiv. Cand divizarea recursiva s-a terminat lista interioara este afisata. Se continua procesarea poligoanelor din lista exterioara.

CATMULL ALG.: Pt suprafete curbate. Se lucreaza in spatiul obiect. Se divide recursiv suprafata pana cand o subsuprafata acopera cel mult un pixel. Un algoritm z-buffer determina daca bucata este vizibila. Se inscrie pixelul cu attributele suprafetei si se afiseaza daca este necesar. Algoritm: pt fiecare petic { se pune peticul pe stiva; cat timp stiva nu e vida { scoate primul petic; daca acopera cel mult un pixel { daca pixelul este cel mai apropiat atunci deseneaza } altfel subdivide peticul in 4 petice si pune-le pe stiva } }.

Z-BUFFER ALG.: Lucreaza cu 2 buffere: unul de cadre (contine culorile si construiește imaginea) – este initializat cu culoarea fundalului si un buffer Z (acelasi nr de intrari, valoarea pe axa Z este stocata pt fiecare pixel) – este initializat cu distanta pana la cel mai indepartat plan de decupaj. Poligoanele sunt prelucrate aleator. Pt poligonul prelucrat, daca punctul de coord (x,y) este mai aproape de observator decat punctul ale carui coord similare si culoare sunt continute in buffere atunci coord si culoarea punctului prelucrat se memoreaza in buffere.

WEILER ALG.: Taie un poligon dupa un alt poligon. Principiu: poligoanele A si B impart scena in 4 regiuni: cuprinse in A, in B, in ambele sau in niciuna. Poligonul care se taie este cel continut in ambele. 1. Initial avem poligoanele A si B. 2. Marcam punctele de intersectie (vor fi varfurile poligonului decupat). 3. Dublam muchiile poligoanelor. 4. Stergem interiorul muchiilor duble a.i. acestea sa fie cursive. Fiecare muchie o notam cu A si B in interior si cu X in exterior. 5. Obtinem 4 regiuni: AX,BX,X si AB. Regiunea AB se decupeaza.

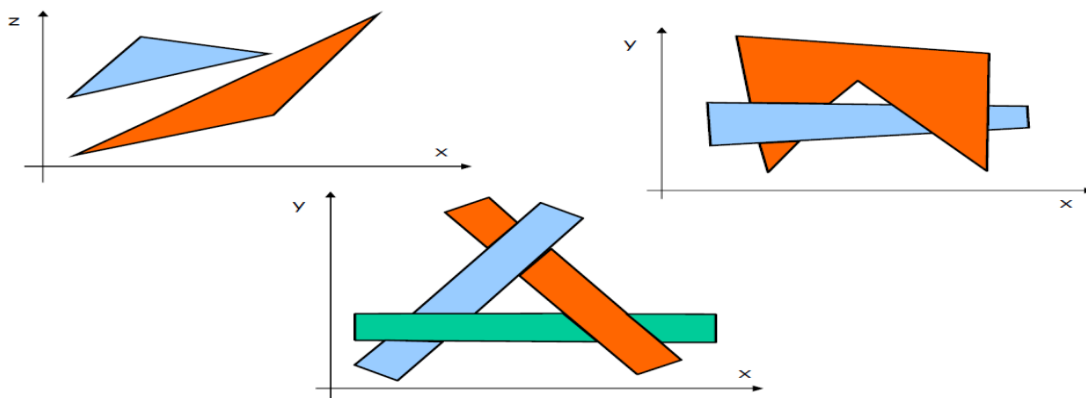
Sortarea in adancime

- cunoscut sub numele de algoritmul pictorului
- deseneaza poligoane in frame buffer in ordinea descrescatoare distantei de la observator.

Algoritmul consta in 3 pasi:

1. Sorteaza toate poligoanele, in conformitate cu cea mai mare(mai indepartata) coordonata z din fiecare poligon(in ordine descrescatoare)
2. Rezolva orice ambiguitate care poate fi provocata cand extensiile poligoanelor z se suprapun, poligoane divizare daca este necesar
3. afisare fiecare poligon in ordine descrescatoare de la cea mai mare coordonata z (adica back to front).

Cazuri de ambiguitate

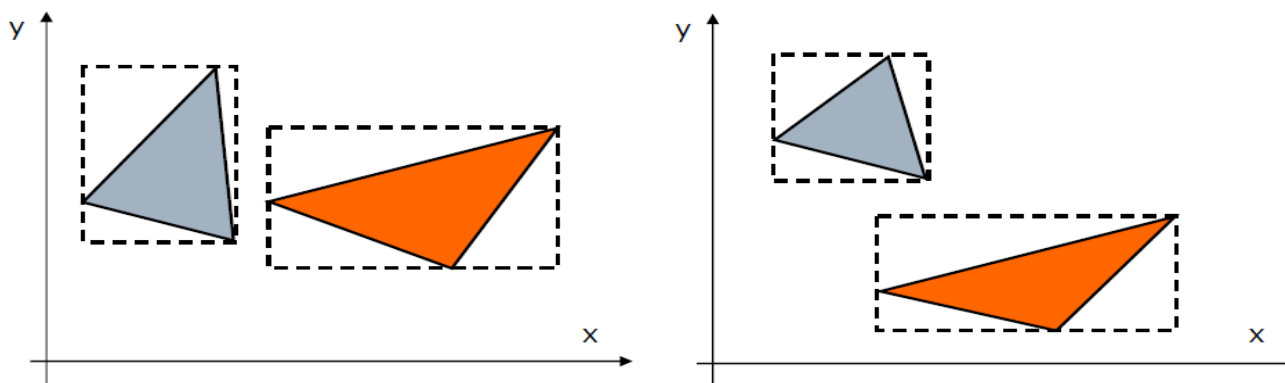


Exista 5 teste :

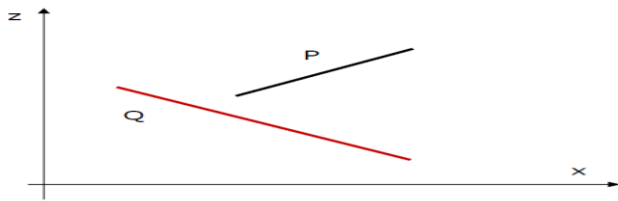
- fie P cel mai departat poligon
 - inainte de incarcarea sa in frame buffer se verifica daca extensia sa nu se suprapune peste extensia altui poligon Q
 - testul ne va spune daca P nu poate ascunde pe Q si ca P poate fi scris in lista inainte lui Q
1. suprapunerea extensiilor poligoanelor pe X.
 2. suprapunerea extensiilor poligoanelor pe Y.
 3. Intregul poligon P se afla de partea opusa la planul Q fata de observator.
 4. Intregul poligon Q se afla de aceeaasi parte a lui P cu observatul.
 5. Proiectiile poligoanelor pe planul (X,Y) nu se suprapun.

Daca toate testele esueaza putem presupune pt moment ca P il mascheaza pe Q si atunci verificam daca il putem pune pe Q in lista inaintea lui P. Se repeta testele 3 4 cu poligoanele inversate. Daca testele reusesc atunci Q se pune primul in lista. Daca esueaza din nou atunci P sau Q trebuie divizat de planul celuilalt. Poligonul nedivizat este inclus in lista pe pozitia corespunzatoare si se continua algoritmul.

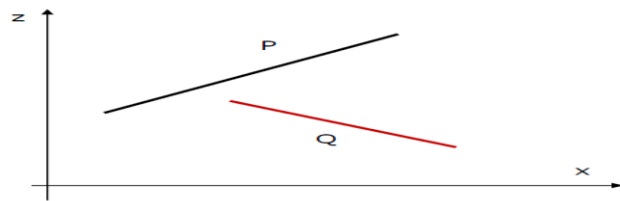
Test 1 si 2 :



Test 3 si 4 :



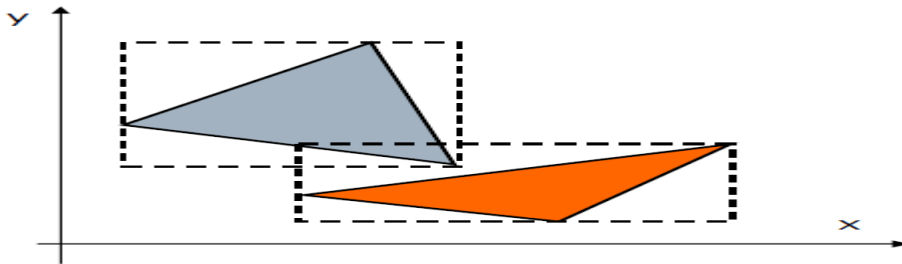
Test 3: draw P, Q



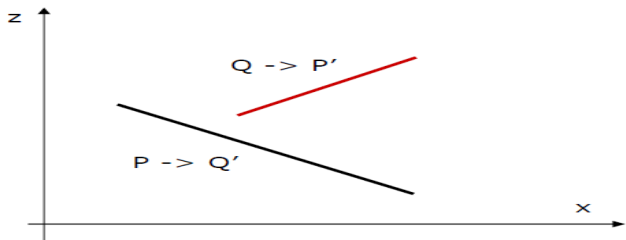
Test 4: draw P, Q

↑
Viewing direction

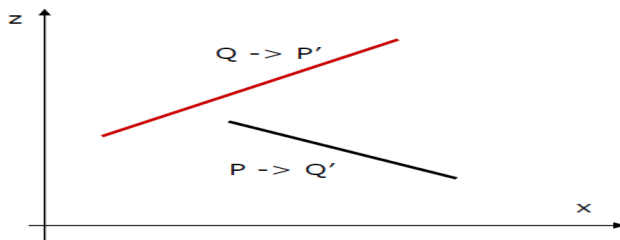
Test 5 :



Test 3 si 4- inlocuire P cu Q :



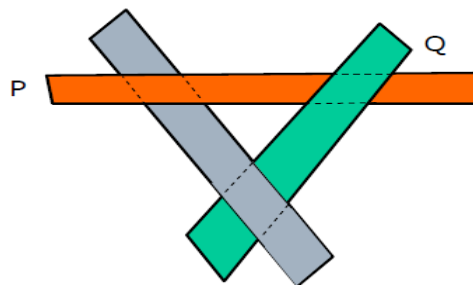
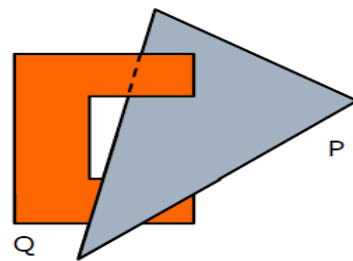
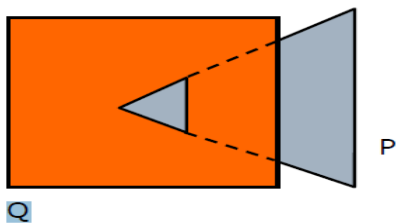
Test 3): test 3 for P and Q fails,
than test 3 for P' and Q', draw P', Q'



Test 4): test 4 for P and Q fails,
Than test 4 for P' and Q', draw P', Q'

↑
Viewing direction

If not successful in box testing and swapping, then split P



ScanLine

- functioneaza cu toate poligoanele care definesc un obiect mai degraba decat un singur poligon la fel ca in Ordered Edge List Algorithm.

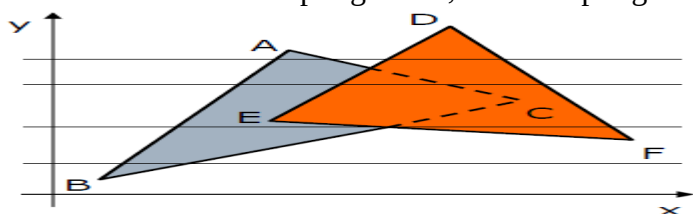
Pasii algoritmului :

1. Se creaza lista muchiilor (ET);
2. Lista poligoanelor (PT);
3. Lista muchiilor active (AET).

1. Lista muchiilor (ET):

- muchiile orizontale se ignora;
- muchiile se sorteaza in grupari in ordine > dupa y-ul minim al muchiei.
- Fiecare grupare se sorteaza > dupa x-ul varfului aflat mai sus.

Fiecare intrare in tabel contine: x-ul varfului muchiei cu y mai mic, y-ul celui alt varf al muchiei, panta muchiei ($(x_2 - x_1) / (y_2 - y_1)$); id-ul poligonului de care apartine muchia si un pointer spre muchia urmatoare din aceeași galeata. 2 – contine id-ul poligonului, varfurile poligonului, culoarea poligonului, un flag initializat la false.



ET:

- AC
- DE
- EF, DF
- AB, BC

x	ymax	Δx	ID	next edge
---	------	------------	----	-----------

2. Lista Poligoanelor (PT)

Contine : id-ul poligonului, coeficienti ecuatiei planului (varfurile poligonului), hasurarea sau culoarea poligonului, un flag initializat la false.

PT:

- ABC
- DEF

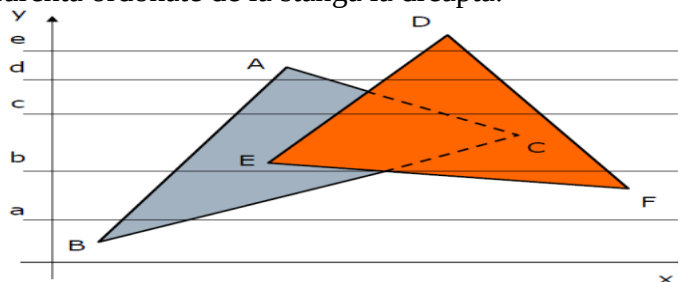
ID	A,B,C,D	Shading info	in-out flag
----	---------	--------------	-------------

3. Lista muchiilor active :

- contine muchiile intersectate de linia de balej curenta ordonate de la stanga la dreapta.

AET:

- a: AB, BC
- b: AB, EF, BC, DF
- c: AB, DE, AC, DF
- d: AB, AC, DE, DF
- e: DE, DF



Scan line parsing :

- parcurgem pixel to pixel, de la stanga la dreapta si actualizam flag-urile muchiilor
- pastram culoarea pana la o intersectie sau pana ies din poligon.
- la intersectie schimb culoarea daca poligonul in care intru e mai in fata.

- algoritmul se poate extinde si pt suprafete generale(suprafete curbe).
- ET este inlocuit de ST(Surface Table), AET de AST(Active Surface Table)
- cand o suprafata este mutata in tabela suprafetelor active este descomupsa intr-un set de poligoane. - - cand iese din tabela setul este descarcat.
- pentru fiecare linie de baleaj se actualizeaza tabela suprafetelor active si pt fiecare pixel de pe linia de baleaj se determina cea mai apropiata suprafata care se proiecteaza in acel pixel.

```

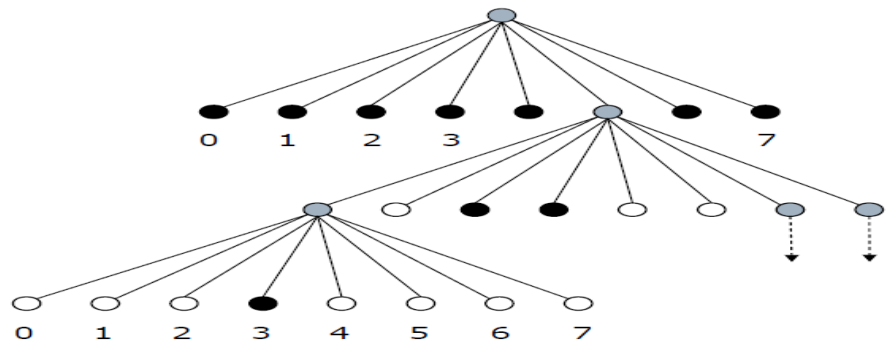
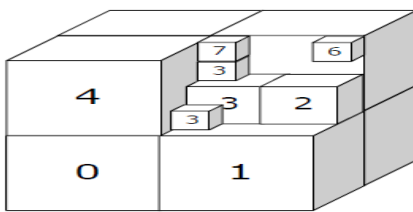
add surfaces to ST;
initialize AST;
for each scan line {
    update AST;
    for each pixel on scan line {
        determine surfaces in AST that project to pixel;
        find closest such surface;
        determine the graphics attribute of the closest surface;
    }
}

```

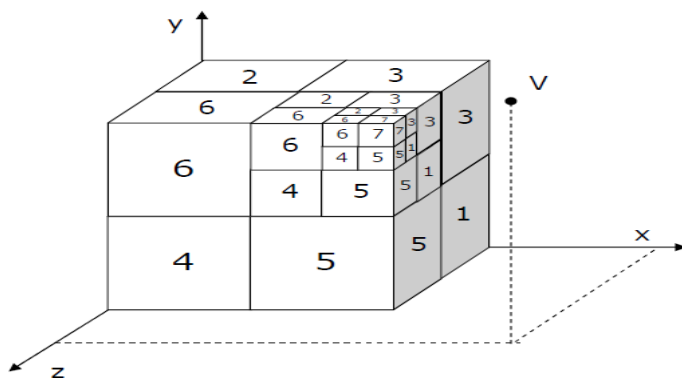
Scan line z-buffer algorithm:

- utlizeaza un z-buffer pentru a rezolva problema suprafetei vizibile
- a single scan line frame buffer and z-buffer is used for each scan line

Octree Based Algorithm:



- structura este compusa din cuburi care nu se intersecteaza.
- Este partial presortat
- se noteaza specific fetele cuburilor pt a accelera algoritmul. –
- Numerotarea feliilor (impartim cubul in 8 cuburi numerotate crescator: pe Oz,Ox,Oy – 0-stanga-fata-jos, 1-dreapta-fata-jos, 2-dreapta-spate-jos, 3-stanga-spate-jos, 4-stanga-fata-sus, etc.) sau numerotare fata-spate (cel mai din spate-0, trei vecini ai lui si cel mai apropiat): 0-stanga-spate-jos,1-dreapta-spate-jos, 2-stanga-spate-sus, 3-dreapta-spate-sus, 4-stanga-fata-jos, etc. Oricare din astea se poate impartin in alte 8 cuburi numerotate la fel.



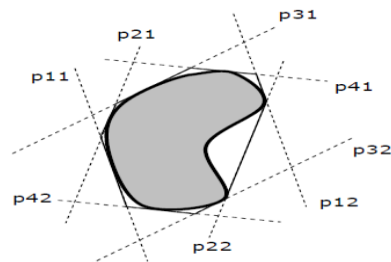
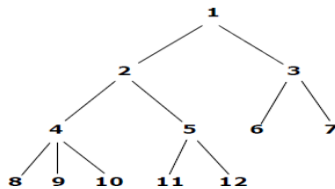
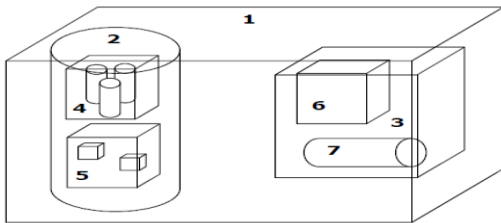
Back to front enumeration and visible faces

z	y	x	Back to front order	Visible faces
-	-	-	7,6,5,3,4,2,1,0	B,D,L
-	-	+	6,7,4,2,5,3,0,1	B,D,R
-	+	-	5,4,7,1,6,0,3,2	B,U,L
-	+	+	4,5,6,0,7,1,2,3	B,U,R
+	-	-	3,2,1,7,0,6,5,4	F,D,L
+	-	+	2,3,0,6,1,7,4,5	F,D,R
+	+	-	1,0,3,5,2,4,7,6	F,U,L
+	+	+	0,1,2,4,3,5,6,7	F,U,R

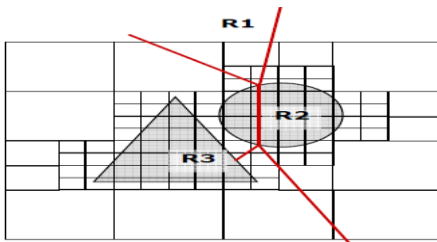
Ray Casting Algorithm:

Aspecte:

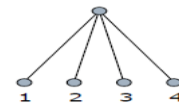
- computing intersections:
 - o Ray(linie parametrica) si un poligon
 - o Linie si cilindru, cub, sfera etc.
 - o Linie si cuandrice(ex : elipsoidala, paraboloid etc)
- Efficiency for visible surface ray tracing :
 - o Optimizare la intersectia de calcul
 - o Evitarea de calcul a intersectiei
 - o Volum ierarhic delimitat
 - o Ierarhie transversala eficienta
 - o Generarea automata a ierarhiei
- computing boolean set operations
 - o calcularea vizibilitati pt reunire 3D, diferenta, intersectie a doua solide
- antialiasing ray tracing
 - o punctul de prelevare a probelor pe o grila ragulata produce aliasing images



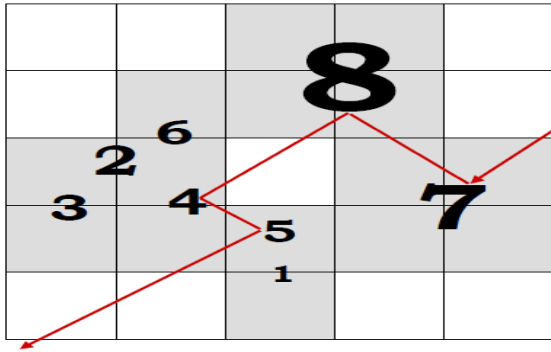
- ☐ Hierarchical bounding boxes
- ☐ Efficient extents



1	2
3	4



- ☐ Quadtree, octree
- ☐ Automatically hierarchy generation
- ☐ Spatial partitioning
- ☐ Efficient hierarchy traversal
- ☐ Optimize the intersection computation



		8	8	
	6	8	8	
23	246		7	7
3	4	15	7	7
		1	8	

- ☐ Automatically hierarchy generation
- ☐ Spatial partitioning
- ☐ Efficient hierarchy traversal
- ☐ Optimize the intersection computation

BSP Trees:

Partitionare spatiului folosind arbori binari.

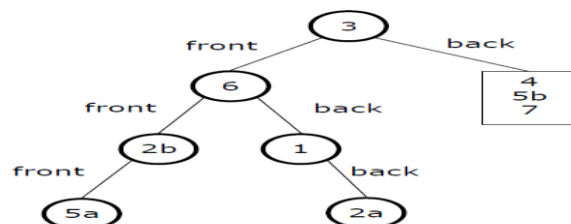
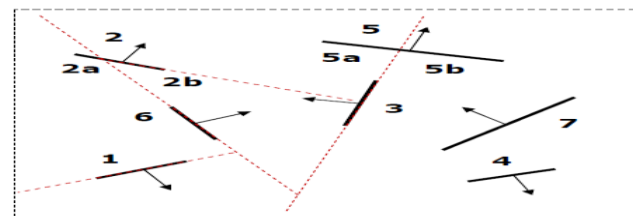
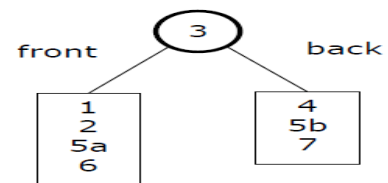
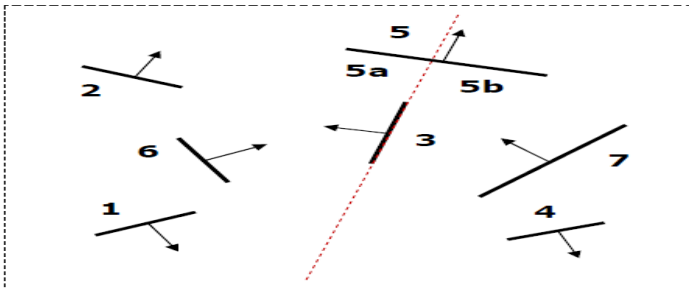
Spatiul e format din gramezi de obiecte separate de plane.

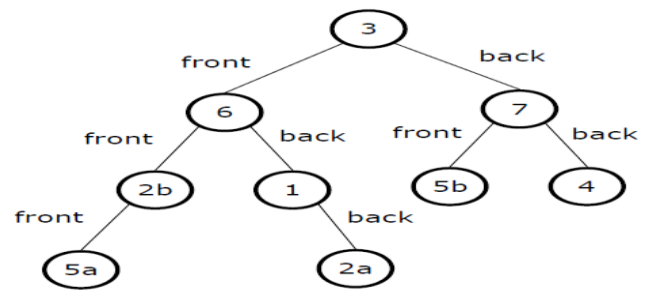
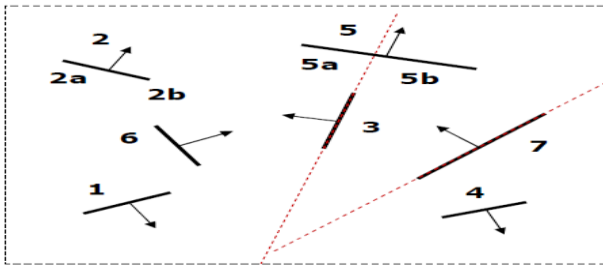
Gramezile aflate de aceeași parte a unui plan cu obs. pot umbri dar nu pot fi umbrite de gramezi din cealaltă parte.

Folositori pt aplicatii in care obs. isi schimba pozitia dar obiectele nu. Construirea arborilor:

1. Se determina planele create de poligoane; 2.
2. Se afla vectorul normal pt fiecare fata a poligonului;
3. Se alege nodul radacina;
4. Se adauga noduri analizand pozitiile relative ale fetelor poligoanelor in raport cu fata poligonului curent.

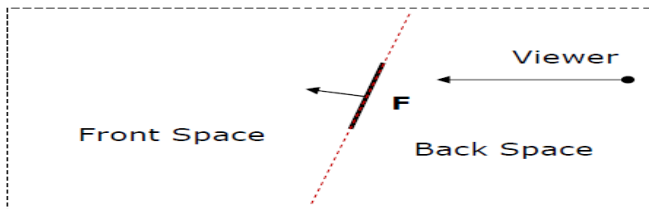
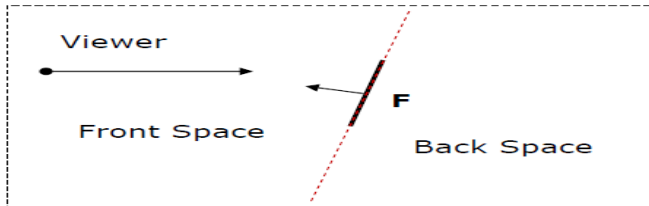
Aleg un nod. Ce vad pun in stanga, ce nu vad pun in dreapta. Daca un poligon e si atunci il impart in doua (5a si 5b). Procedez la fel cu un nod din lista din stanga si cu unul din lista din dreapta, dar ma raportez doar la nodurile din aceeași lista. Continui pana cand nu mai exista liste ci doar noduri. Ordinea afisarii: daca nodul e orientat spre obs atunci: spate-nod-fata, altfel: fata-nod-spate.





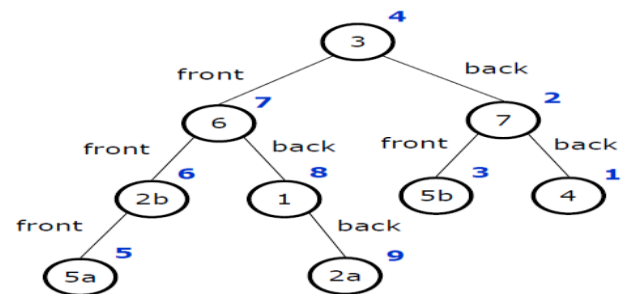
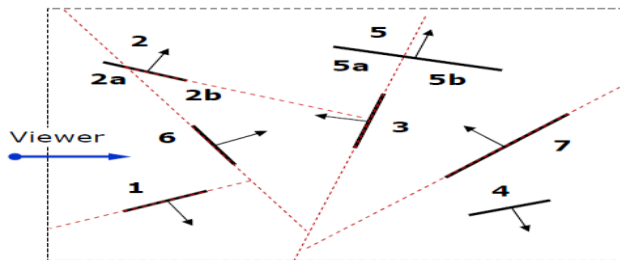
Display order:

1. Back space polygons
2. Root face
3. Front space polygons



Display order:

1. Front space polygons
2. Root face
3. Back space polygons



Display order: 4, 7, 5b, 3, 5a, 2b, 6, 1, 2a

REPREZENTAREA OBIECTELOR 3D

Tipurile de reprezentare:

- **Funcție implicită:** obiectul este o funcție analitică (ex sferă: $x^2+y^2+z^2=r^2$)
- **Poligonal:** obiectul este aproximat cu un set de planuri poligonale.

Obiectul 3D este alcătuit din mai multe suprafețe, suprafețele din mai multe poligoane și poligoanele din mai multe noduri. Acuratețea obiectului depinde de numărul de poligoane. **Avantaje:** omniprezent în grafica pe calculator, simplă, directă, formă intermediară pt alte structuri de date. **Dezavantaje:** aproximarea suprafețelor curbate, precizia depinde de nr. De poligoane, maparea texturilor, finetea suprafețelor, algoritmi pt. umbre.

Modelare pe baza de varfuri:-pentru fiecare varf se țin minte coordonatele, iar pentru fiecare poligon varfurile

Modelare pe baza de poligone:-reprezentare prin liste (lista obiectelor->lista suprafețelor: lista de varfuri(coordonate+ normală) și lista de normale). Dezavantaj: Fiecare muchie va fi parcursă de două ori

Model pe baza de muchii:- se țin coordonatele varfurilor, varfurile muchiilor, muchiile fețelor. Lista muchii: un element cuprinde pointeri către vf. și normale

Structuri de date la modelare pe baza de poligonală:

- **Object Array:** un tablou de toate suprafețele al obiectului 3D modelat cu poligoane. O poziție al tabloului este o listă de indici la componente poligonale.
- **Surface Array:** este un set de poligoane care descriu suprafața. O poziție în tablou descrie un poligon ca o listă de indici la nodurile poligonului și un index la normale poligonului.
- **Vertex Array:** descrie informația care este asociată la fiecare nod. Contine trei coordonate și un normal vector care descrie nodul. Normal vectorul are trei componente N_x, N_y, N_z .
- **Polygonal Array:** un set de vectori normali al poligonului. Fiecare vector are trei componente.

Scanarea obiectelor: laser, tomografic, viziunea calculatorului.

Reprezentare triunghiulară (sunt mereu planare, hardware-ul este mereu optimizat pentru triunghiuri): Lists(3 muchii/triunghi), Fans(avantaj 1-5), Strips(fasii, 1-5).

- **Bicubic parametric patches:** obiectele sunt reprezentate ca un set de patch-uri. Seturile de patch-uri sunt polinomiale în două variabile parametrice *ex: Bezier, Hermite, B-spline).

$$P(u,v)=[x(u,v), y(u,v), z(u,v)], 0 \leq u \leq 1, 0 \leq v \leq 1$$

Curbe: Hermite, Bezier, Spline, Uniform B-Spline

Suprafețe: Hermite, Bezier, B-Spline

Avantaje: este exact și economic pentru că este analitică. Cu un obiect poligonal acuratețea formei poate fi aproximat numai cu un cost suplimentar la procesare.

- **Constructive solid geometry (CSG):** obiectul este o colecție de obiecte elementare (sferă, cilindru, cub).
Modelare ierarhică bazată pe structură de arbori. Nodurile interne sunt un set de operații: union, intersecție sau diferență. La muchii sunt asociate transformări (scalare, rotație, translație). Frunzele conțin numai geometrie. Permite forme complexe cu câteva primitive (cilindru, cub). Motivată de design-ul ajutat de calculator și manufactură. Formatul este folosit în CAD.
- **Tehnica diviziei de spațiu:** obiectul este incorporat în spațiu unde zonele sunt etichetate în funcție de obiect. (Octree, Bsp-Tree)
- **Modelare procedurală:** se aplică reguli algoritmice pentru a genera forme, fractali.

Sisteme pe baza de particule: descrie forme naturale și iregulare care sunt fluide (fluidlike), obiecte cu particule care se modifică în timp. Contine unul sau mai multe particule. Fiecare particulă are următoarele atribute stohastice: poziție, viteză, direcție, culoare, durată de viață, vârstă, formă, dimensiunea, transparent. Lindenmayer – System –regulile de producere: $W \rightarrow F \mid F \rightarrow FF+[F-F-F]-[F+F+F]$.

Ciclu de viață a particulelor: 1.Generare: generat random într-o locație definită. 2.Dinamica particulelor: atributele particulelor se schimbă în timp 3.Extincția: când vârsta va fi egală cu durata de viață. 4.Reprezentare: pentru simplificarea procesului se folosesc presupuneri

Curs 5: TRASAREA (rendering)

Este o colectie de operatii necesare pt a proiecta o vedere a unui obiect sau a unei scene pe o suprafata de vizualizare. Strategiile depind de reprezentarea obiectelor si de algoritmi existenti. Depinde de model: parametric, poligonal, CSG.

Trasarea poligoanelor – procesam poligonul ca o entitate singulara; input: lista de poligoane, output: culoarea fiecarui pixel de pe ecran.

3D Objects rendering pipeline: Spatiul de coord locale (definirea obiectului, se stocheaza in baza de date folosind coord varfurilor) ->transf de modelare-> Spatiul de coord globale (transformari 3D, compunerea scenei, luminilor si a punctului vizual de referinta) -> transf de vizualizare-> Spatiul de vizualizare (eliminarea poligoanelor care nu sunt vizibile din punctul de vizualizare, eliminarea partilor din scena care sunt in afara volumului de vizualizare) -> proiectie -> Spatiul ecran 3D (eliminarea suprafetelor ascunse, umbre, anti-aliasing, rasterizare) -> Display space.

Operatii:

- Maparea texturilor: fixa (in toate sistemele: local, global, view si screen) si dependenta de obs. (view si screen)
- Calcularea umbrelor: depinde doar de pozitiile obiectelor si a surselor de lumina, shadow volumes si polygons, spatiu local, global si de vizualizare
- Miscarea observatorului: modificari in sp. global
- Modif. pozitiei sau a dimensiunii obiectelor – sp. global si de vizualizare
- Modif. atributelor grafice ale obiectelor – in toate spatiile

Trasarea suprafetelor parametrice: se transforma in suprafete poligonale si se aplica trasarea poligonala. In general nr poligoanelor este mai mare decat nr peticelor.

Pasii conversiei:

1. Se converteste suprafata in forma Bezier prin inmultire matriceala.
2. Se imparte suprafata in petice prin curbe isoparametrice pana cand un criteriu de netezire este satisfacut. Algoritmul De Casteljau: Curba este definita de 4 puncte de control. Unim cele 4 puncte cu 3 linii si le marcam mijlocul la fiecare (obtinem alte 3 puncte). Unim cele 3 puncte cu doua linii si marcam mijlocul lor (inca 2 puncte). Unim cele 2 puncte si marcam mijlocul liniei. Acesta este unul din punctele care definesc curba bezier. Pentru a calcula celelalte puncte folosim o interpolare liniara, in loc de a lua mijlocul liniilor.

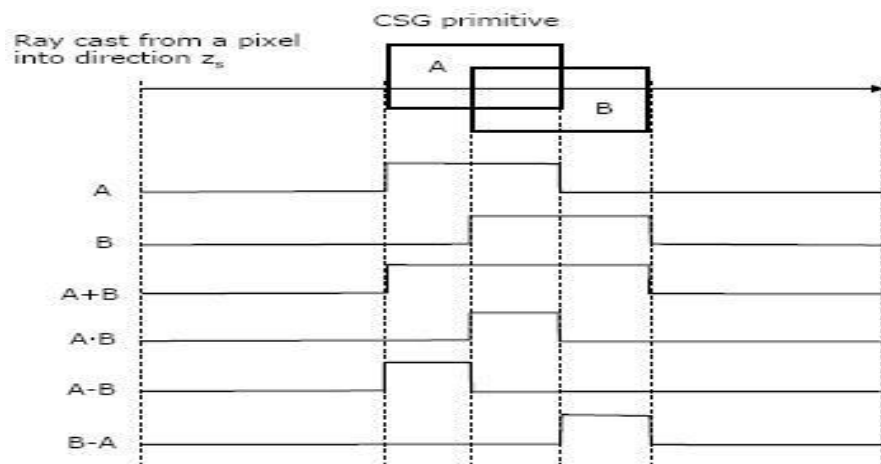
Trasarea CSG – nu este o reprezentare cu frontiere (care sa imparta spatiul in regiuni interne si externe ale unui obiect). Pentru aceasta se pot folosi:

1. CSG ray tracing
2. Conversia la o reprezentare voxel (volumetric pixel) urmata de volume rendering (repr unui volum)
3. Using a version of the Z-Buffer algorithm.

CSG RAY TRACING – reduce problema la o dimensiune, considera pt fiecare pixel o raza vizuala care uneste obs. cu pixelul. Se baleiaza spatiul de obiecte cu un set de raze paralele. Doua etape:

1. Consideram o raza unica – verificam fiecare primitiva daca este intersectata de raza; intersectiile sunt sortate pe axa Z si exista o clasificare raza/primitiva pt fiecare raza; verificam orice combinatie booleana intre primele doua primitive intalnite de raza.
2. Alocam o valoare de umbra pixelului printr-un model de reflexie aplicat la prima intersectie a razei de lumina.

Operatiile au loc in spatiul ecranului. Ca la diagrame de timp. Cand nu trece prin obiect e pe 0, cand trece prin obiect e pe 1, pe directia Oz. Punem una sub alta diagramele si afisam altele pt interesectione, reuniune, diferenta:



Ray Tracing integreaza intr-un singur model: reprezentarea frontierelor dintr-un model CSG, eliminarea suprafetelor ascunse si umbrele.

POLYGONAL OBJECTS RENDERING(1)

Basic render –abordarea traditionala pentru renderizarea obiectelor 3D

Polygonal render –procesarea unui poligon ca o singura entitate

-avand la intrare o lista de poligoane va rezulta culoare pentru fiecare pixel de pe ecran

-Avantaje: modelarea este straightforward, informatia geometrica este pastrata la nodurile poligonului, poate fi utilizata pentru renderizarea obiectelor formate din seturi de patch-uri bicubice

-Dezavantaje: este dificil de mapat texturi dintr-un domeniu 2D pe un obiect, algoritmi pentru umbre bazati pe obiecte poligonale produc umbre „hard-edge”, poligoanele sun costisitoare

Sistemul de coordonate locale: se stocheaza nodurile, normalele poligonului si normalele nodurilor

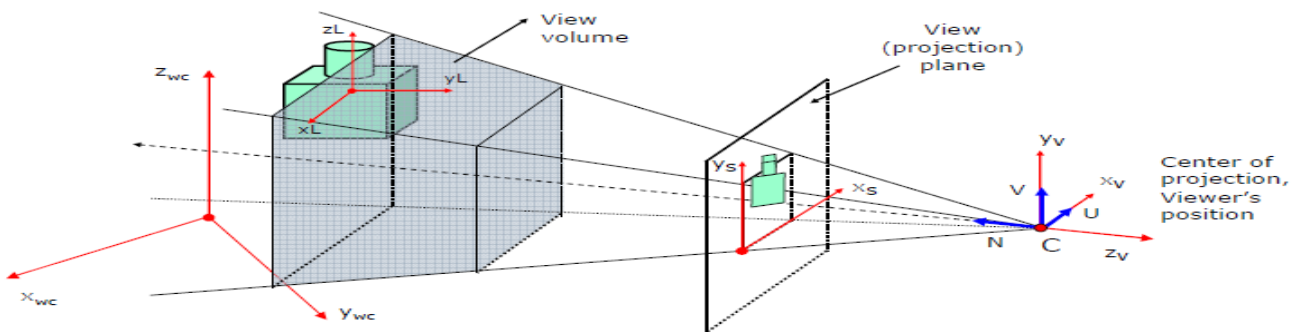
Sistemul de coordonate globale: obiectele sunt plasate in scena, plasarea unui obiect in scena reprezinta transformarea necesara pentru a muta obiectul din sistemul local in sistemul global. Apoi se instantiaza obiectele si se definesc surse de lumina si attributele suprafetelor

Spatiul de vizualizare:

-se stabilesc parametri si volumul de vizualizare

-se defineste pozitia observatorului (camerei), poate fi asezata oriunde si indreptata in orice directie

-planul de vizualizare-planul pe care se proiecteaza scena.



3 vectori si o pozitie:

-C –pozitia camerei, este si centrul proiectiei

-N –vectorul normal al planului, de- a lungul axei z

-V – reprezinta axa y, orienteaza camera in directia de vizualizare

-U –vector optional, reprezinta axa x

Trecerea din spatiul lumi reale in spatiul de vizualizare se face dupa urmatoare ecuatie:

$[x_v \ y_v \ z_v \ 1] = [x_w \ y_w \ z_w \ 1] * H$, unde $H = T * R$, T este translatia la punctul C, iar R este rotatia in sistemul de vizualizare:

1 0 0 0

T = 0 1 0 0

0 0 1 0

-xC -yC -zC 1

,R mapeaza U,V si N la vectorii unitate :

U -> (1, 0, 0)

V -> (0, 1, 0)

N -> (0, 0, -1)

Vizibilitate:

Pt a determina fetele vizibile se face $N_p \cdot L$, unde N_p este vectorul normal al fetei si L este vectorul de vizualizare.

Cazuri de vizibilitate:

- Daca unghiul <90 atunci fata e vizibila.

- Daca unghiul $=90$ atunci observatorul este in poligon .
- Daca unghiul <90 atunci fata nu e vizibila.

Putem accelera testul de vizibilitate prin compararea semnelor vectorilor normala si de vizualizare.

Spatiu ecran:

Spatiul ecran este spatiul 3D. Ordinea punctelor din spatiul real se pastreaza si in spatiul ecran, doar distantele se modifica ($z_v=S$, $x_s=S*x_v/z_v$, $y_s=S*y_v/z_v$), $X=x_v$, $Y=y_v$, $Z=z_v$, $w=z_v/S$. Proiectia perspectiva devine proiectie ortogonala ($x_s=x_v$, $y_s=y_v$, $z_s=0$). In proiectia ortogonala planul de vizualizare este perpendicular pe directia de proiectie.

Volumul de vizualizare:

Volumul de vizualizare este spatiul de interes din spatiul de vizualizare. Este definit de 6 plane: $z_v=S$, $z_v=F$, $x_v=+-h*z_v/S$, $y_v=+-h*z_v/S$.

$2h$ =dimensiunea planului de vizualizare,

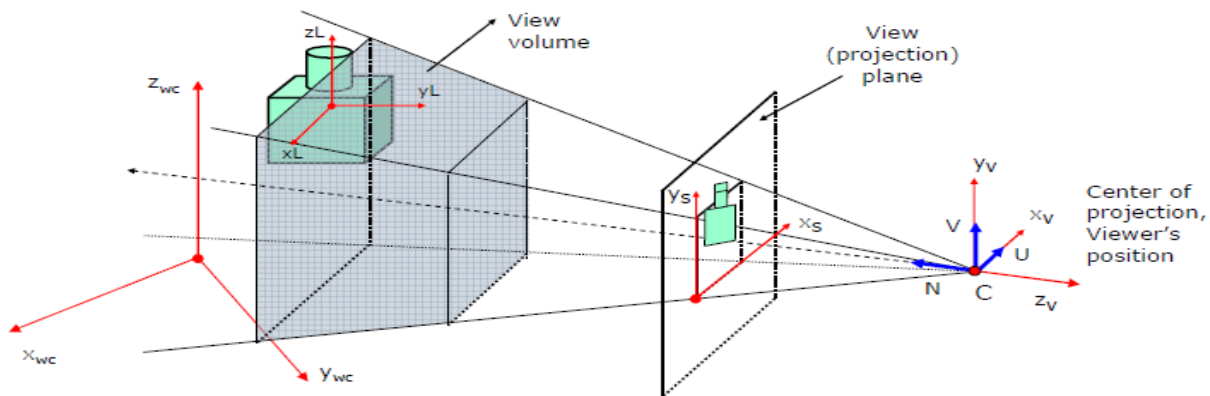
S =distanța pana la planul de vizualizare, distanța pana la planul ecranului virtual si distanța pana la planul apropiat

F =distanța pana la planul indepartat.

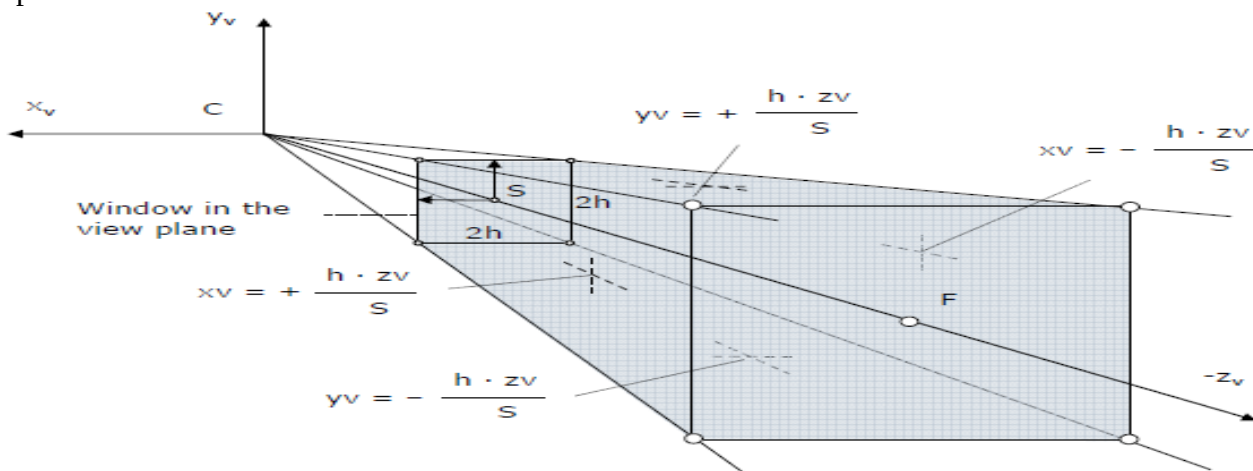
S si F sunt parametri de baza ai aplicatiei

POLYGONAL OBJECTS RENDERING(2)

Modelul conceptual al procesului de vizualizare 3D:



Spatiul ecranului este definit in interiorul volumului de vizualizare:



Transformarea in spatial ecranului:

Spatiul ecranului este un spatiu 3D.

Condițiile transformării: Z-urile trebuie să fie normalizate pentru ca precizia să fie maximă, Liniile din spațiul vizualizării trebuie să fie transformate în linii din spațiul vizualizării, planurile din spațiul vizualizării trebuie să fie transformate în planuri din spațiul vizualizării.

Pentru a satisface condițiile transformarea lui z ia forma:

$$zs = A + B/zv$$

unde A și B sunt constante având următoarele constrângeri:

Alegând $B < 0$ astfel încât zv să crească atunci și zs crește. Aceasta perpetuează notația intuitivă a adâncimii. Dacă un punct se află în spatele altuia, atunci va avea o valoare zv mai mare. Dacă $b > 0$ va avea de asemenea o valoare zs mai mare.

Normalizăm intervalul de valori al lui zs a.i. valorile lui zv să aparțină intervalului $[S, F] \Rightarrow zs \in [0, 1]$.

Transformarea din perspectivă este dată de relația:

$$h * zv / S > 1 \text{ atunci } xv \rightarrow xs$$

atunci coordonatele ecranului sunt:

$$xs = xv * S / h * zv; \quad ys = yv * S / h * zv.$$

În final transformarea este dată de relațiile:

$$xs = S * xv / h * zv;$$

$$ys = S * yv / h * zv;$$

$$zs = F * (1 - S / zv) / S - F.$$

Coordonate (X, Y, Z, W) sunt numite coordonate omogene. W păstrează informații despre înălțime, necesare pentru operațiunile grafice de mai târziu.

Matricea de transformare în perspectivă poate fi concatenată cu alte matrici de transformări precedente pentru a defini o secvență de transformări numită “rendering pipeline”.

$$\begin{bmatrix} X & Y & Z & w \end{bmatrix} = \begin{bmatrix} xv & yv & zv & 1 \end{bmatrix} T_{pers}$$

$$T_{pers} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{h \cdot F}{S(S - F)} & -\frac{h}{S} \\ 0 & 0 & -\frac{h \cdot F}{S - F} & 0 \end{bmatrix}$$

Transformarea în perspectivă este singura transformare care folosește coordonata W.

Matricea standard de transformare Mij pentru scalare, rotație și translație are elementul M33=1. W este 1 și rămâne 1 și după transformare.

Algoritmul Z-buffer

Avantaje: Implementare simplă, Procesează modelele bazate pe poligoane, Nu necesită nici o preprocesare a poligoanelor, Procesează poligoanele în orice ordine sunt luate din modelul graphic

Dezavantaje: Calculele pt umbre sunt realizate și pentru pixelii ascunși, Necesită resurse mari de memorie, Timpul de calcul crește în cazul proiecțiilor de perspectivă, Dimensiunea bufferului Z depinde de acuratețea cu care este salvată adâncimea pentru fiecare punct.

Calculul intensității luminii și al culorii a unei suprafețe poligonale se realizează cu două tehnici de bază: Interpolarea Gouraud, Interpolarea Phong.

Umbrele Gouraud: Calculează intensitatea luminii pt punctele din interiorul poligonului, Consideră sursa de lumină la infinit

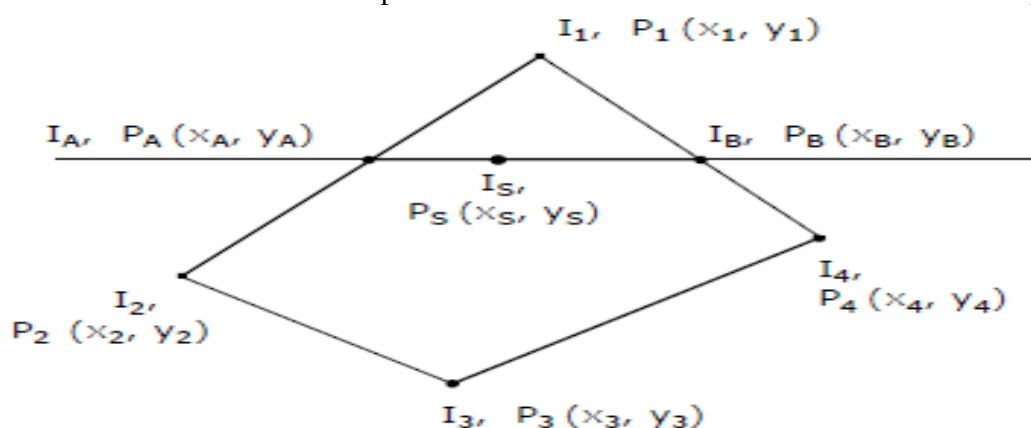
Fiecarui vertex îi este asociat un vector care ar trebui să fie ortogonal față de suprafața curbă inițială

Normalul vertexului poate fi dat sau calculat din normalele poligoanelor învecinate.

Pasii algoritmului:

Calculul intensității luminii I_v pt fiecare vertex. Intensitatea luminii este o funcție a vectorului normal și a poziției lui relative față de sursa de lumină și de poziția privitorului.

Pentru fiecare poziție a liniei de scanare calculate de interpolator, intensitatea luminii I_a și I_b în punctele A și B. Distanța dintre ele este dată prin intersectarea liniei de scanare cu 2 muchii ale poligonului.



Pentru fiecare poziție a punctului S se calculează intensitatea I_s folosind I_a și I_b .

$$I_A = I_1 + (I_2 - I_1) / (Y_2 - Y_1) \cdot (Y_A - Y_1)$$

$$I_B = I_1 + (I_4 - I_1) / (Y_4 - Y_1) \cdot (Y_B - Y_1)$$

$$I_S = I_A + (I_B - I_A) / (X_B - X_A) \cdot (X_S - X_A)$$

Interpolarea Phong

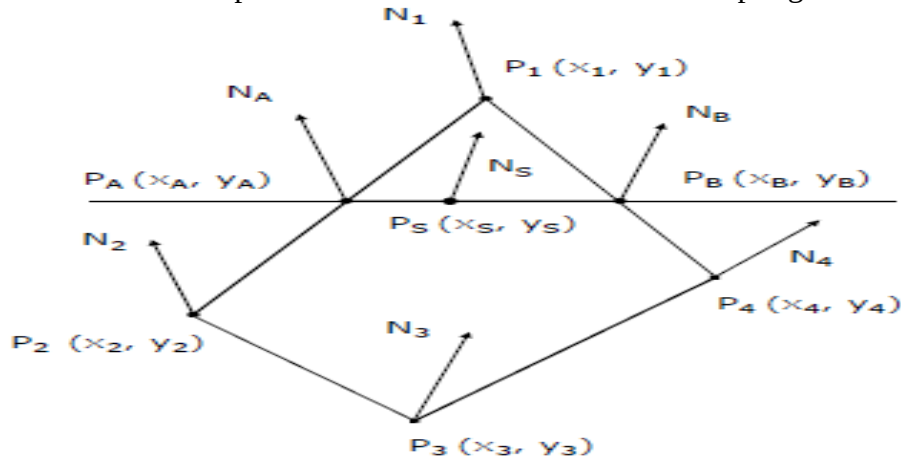
Considera sursa de lumina la infinit.

Calculeaza intensitatea luminii pt punctele din interiorul poligonului.

Atributele interpolate sunt normalele vertexilor nu intensitatile acestora.

Fiecarui vertex ii este asociat un vector care ar trebui sa fie ortogonal fata de suprafata curbata initiala

Normalul vertexului poate fii dat sau calculat din normalele poligoanelor invecinate.

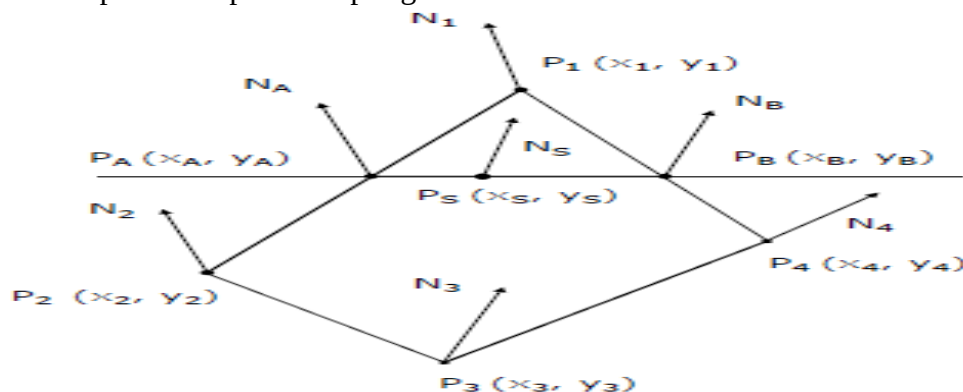


Pasii algoritmului:

Calculul vectorului normal Nv pt fiecare vertex.

Pt fiecare pozitie a liniei de scanare se calculeaza prin interpolare vectorii normali Na si Nb situati la punctele A si B. Distanța este data prin intersectia unei linii de scanare cu 2 muchii ale poligonului.

Na si Nb sunt folositi pt a interpola pe Ns. Folosind o astfel de abordare se calculeaza un vector normal pt fiecare punct sau pixel din poligon.



$$N_A = N_1 + (N_2 - N_1) \frac{Y_A - Y_1}{Y_2 - Y_1}$$

$$N_B = N_1 + (N_4 - N_1) \frac{Y_B - Y_1}{Y_4 - Y_1}$$

$$N_S = N_A + (N_B - N_A) \frac{X_S - X_A}{X_B - X_A}$$

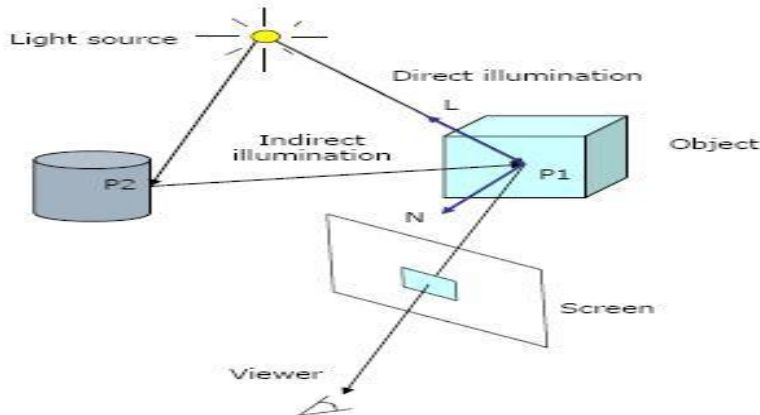
Curs 8: LUMINI

Modelul de reflexie:

- descrie interaciunea dintre o suprafață și o sursă de lumină
- calculează intensitatea luminii în punctele de pe obiectul 3D, lucrează în coord. globale
- poate fi local (pct de pe suprafața și sursă de lumină) sau global (se consideră și lumina reflectată de pe alte obiecte).

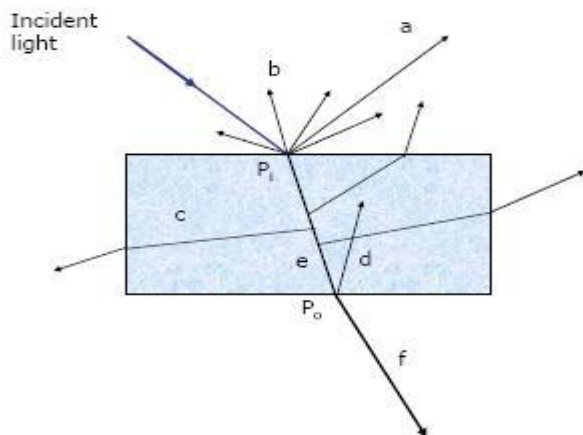
Modelul de iluminare definește natura luminii emise.

Modelul incremental lucrează în spațiul ecran și calculează intensitatea luminii într-un pixel din planul de proiecție.



Interacțiunea luminii cu un obiect 3D:

- reflectie speculară
- reflectie difuză
- emisie fluorescentă
- reflectie internă
- Absorbție
- lumina transmisă (refracție)



Lumina incidentă = lumina reflectată + lumina difuză + lumina absorbită + lumina refractată.

Intensitatea luminii și a energiei:

- Bidirectional Reflectivity Function (BRDF): descrie calitativ fenomenul de reflectie.

Definiție: $R_{bd}(\lambda, \theta_i, \Phi_i, \theta_v, \Phi_v)$;

- (θ_i, Φ_i) direcția luminii care vine

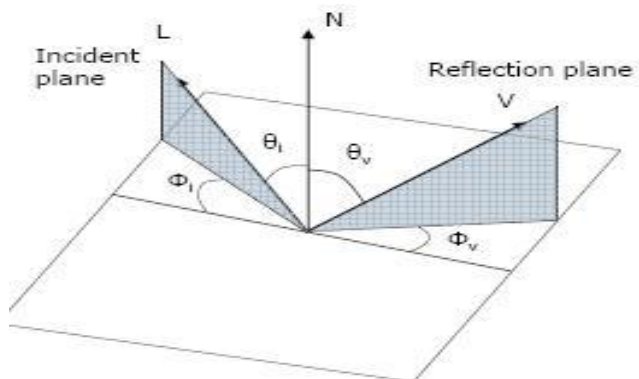
- (θ_v, Φ_v) direcția luminii care pleacă

- Modelul de reflectie vs Metoda de radiație: $I(\theta_v, \Phi_v) = I_{emisa}(\theta_v, \Phi_v) + I_{reflectata}(\theta_v, \Phi_v)$

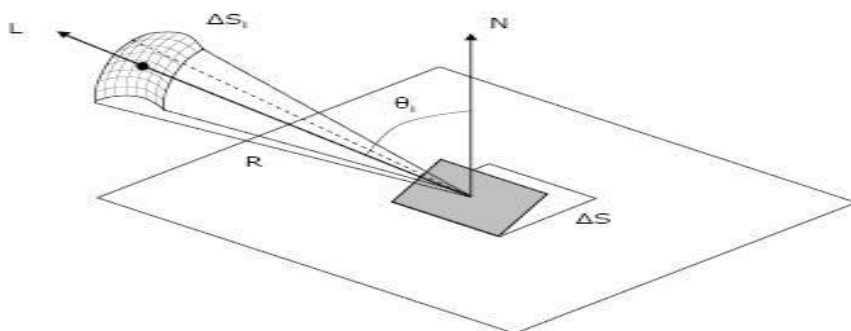
$R_{bd}(\lambda, \theta_i, \Phi_i, \theta_v, \Phi_v) = I_v(\theta_i, \Phi_i, \theta_v, \Phi_v) / E_i(\theta_i, \Phi_i)$

- $E_i(\theta_i, \Phi_i)$ energia luminii care vine

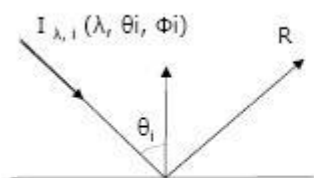
- $I_v(\theta_i, \Phi_i, \theta_v, \Phi_v)$ intensitatea luminii care pleacă



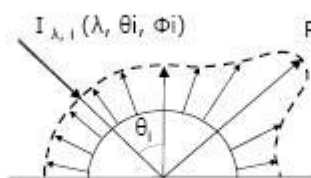
- energia care vine: $E_i(\theta_i, \Phi_i) = I_i(\theta_i, \Phi_i) \cos \theta_i d\omega_i$, unde ω_i este unghiul solid $d\omega_i = \Delta S_i / R^2$



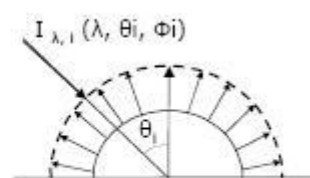
- BDRF este impartit in 3 componente: contributia speculara, difuza directionala (datorita iluminarii directe) si difuza ideala (dat. reflectiilor multiple si intersectiile suprafetelor)



(a) Specular reflection



(b) Directional diffuse reflection



(c) Ideal diffuse reflection

Modelul lui Phong de reflectie locala: Considera doar iluminarea directa. Este o combinatie lineara de 3 componente: ambientala, speculara si difuza.

$$I_{\lambda,r} = I_{\lambda,ambient} k_{ambient}(\lambda) + I_{\lambda,i} k_{diffuse}(\lambda) (N \cdot L)$$

- Componenta ambientala este o aprox a luminii globale; nu are directie, este constanta in toata scena, este independenta de pozitia obiectelor. Suprafetele determina cata lumina ambientala este reflectata.

Legea cosinusului a lui Lambert: energia reflectata de pe o suprafata mica intr-o anumita directie e proportionala cu cosinusul unghiului dintre acea directie si normala suprafetei: $I_{diffuse} = I_i * k_d(\lambda) * \cos \theta$

- Componenta speculara depinde de cat de lucioasa este suprafata. Ideala este oglinda. $I_{specular} = I_i * k_s * (\cos \phi)^n = I_i * k_s * (R \cdot V)^n$. Cu cat n e mai mic cu atat suprafata luminata e mai mare.

Modelul de reflectie Phong:

$$I_v(\lambda) = I_a * k_a(\lambda) + I_i * k_d(\lambda) * (L \cdot N) + I_i * k_s * (R \cdot V)^n, \text{ unde } L \cdot N = \cos \theta; R \cdot V = \cos \phi.$$

Modelul de implementare Phong:

$$I_v(\lambda) = I_a * k_a(\lambda) + I_i * k_d(\lambda) * (L \cdot N) + I_i * k_s * (N \cdot H)^n, \text{ unde } L \cdot N = \cos \theta; N \cdot H = \cos \beta.$$

Culoarea in modelul Phong: este o functie de lungime de unda a intensitatii luminoase. Culoarea obiectelor este controlata de coeficientii difuzi, iar culoarea scenei de coeficientii ambientali.

$$I(r,g,b) = I_a * k_a(r,g,b) + I_i [k_d(r,g,b) * (L \cdot N) + k_s * (N \cdot H)^n].$$

Sursele de lumina sunt punctiforme; sunt localizate la infinit la fel ca si obs; componenta difuza si speculara sunt modelate local; culoarea reflexiei speculare este aceeaasi cu a sursei de lumina; contributia ambientala e constanta.

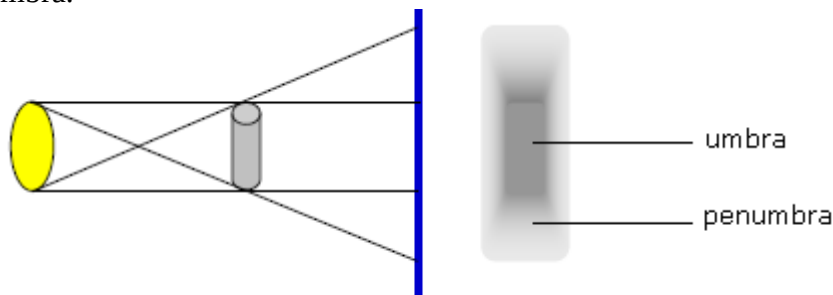
UMBRE

Procesarea umbrelor este cea mai utilizata modalitate de realizare a fotorealismului. Aceasta se refera la procesarea poligoanelor de umbra.

Aspecte ale umbrelor:

- tipul sursei de lumina: punctiforma, directionala (spot) ;
 - tipul umbrei : umbre soft sau umbre hard;
 - tipuri de proiectie : pe sol, pe orice alt plan, pe obiectele vecine;
- Umbra**-portiunea din umbra care este complet in afara razei de lumina.

Penumbra-suprafata care primeste o parte din lumina si realizeaza trecerea graduala de la umbra la penumbra.



Pentru procesarea umbrelor avem 2 categorii de algoritmi:

- algoritmi care proceseaza iluminare globala (Ray Tracing, Radiosity);
- algoritmi de iluminare locala (shadow projected, scan line, shadow volum, Z-buffer);

Umbre pe sol(ground plane)(iluminare locala)

Acest tip de umbre ea in calcul cazul unei scene cu un singur obiecte, sau scene cu mai multe obiecte izolate astfel incat nu isi fac umbra unul celuilalt. Pentru fiecare poligon calculam umbra proiectata de fiecare raza de lumina pe planul bazei ($z=0$).

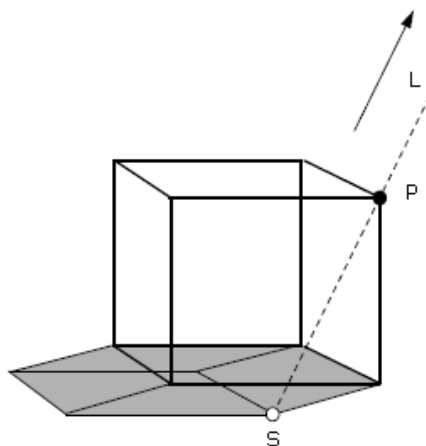
Avem doua abordari pentru acest tip de umbre: o abordare in care consideram sursa de lumina la infinit si una in care sursa de lumina este locala.

1) Sursa la infinit: Daca sursa e la infinit, trasam linii prin toate punctele obiectului pe podea si umplem poligonul rezultat cu umbra; toate razele sunt paralele.

Consideram 2 puncte: (x_l, y_l, z_l) sursa de lumina si un punct (x_p, y_p, z_p) al obiectului. Trebuie sa determinam $S(x_s, y_s, z_s)$ care este punctul umbra si pentru care $z_s=0$.

Pornim de la ecuatia unei drepte determinata de un vector si un punct:

$$S = P - \alpha L$$



Dar $z_s = 0$:

$$0 = z_p - \alpha z_l$$

$$\alpha = z_p / z_l$$

si

$$x_s = x_p - (z_p / z_l) x_l$$

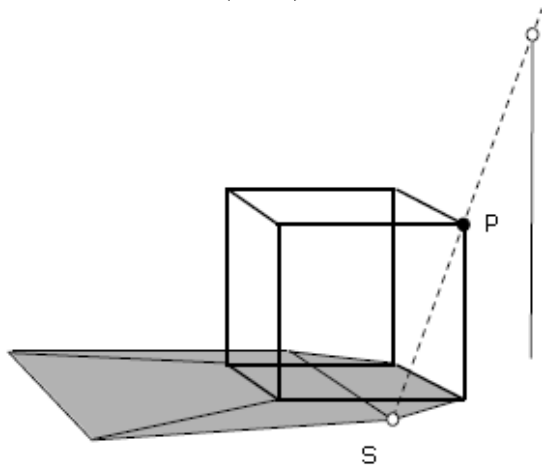
$$y_s = y_p - (z_p / z_l) y_l$$

2) Sursa locala: Daca sursa e locala, razele converg din punctul sursei.

Pornim prin a considera 2 puncte: (x_l, y_l, z_l) sursa de lumina si un punct (x_p, y_p, z_p) al obiectului. Trebuie sa calculam punctul $S(x_s, y_s, 0)$.

Pornim de la ecuatia unei drepte determinata de un vector si un punct:

$$S = P + \alpha(P - L)$$



Dar pe sol $z_s = 0$

$$0 = z_p + \alpha(z_p - z_l)$$

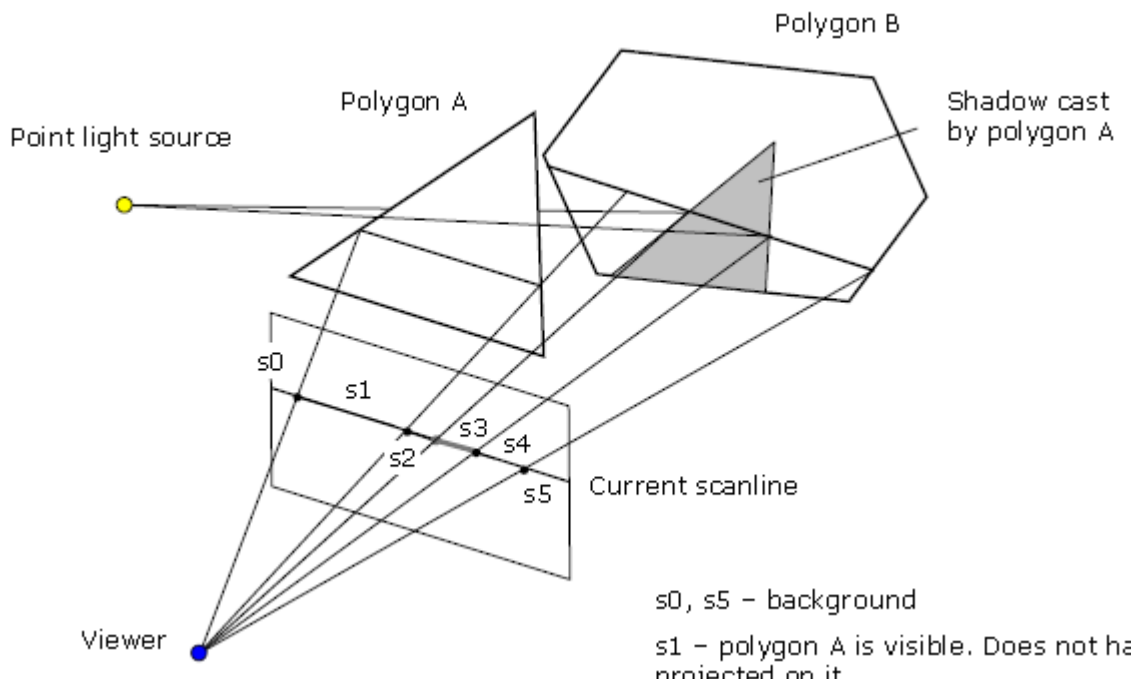
$$\alpha = -z_p / (z_p - z_l)$$

Deci:

$$x_s = (x_l z_p - x_p z_l) / (z_p - z_l)$$

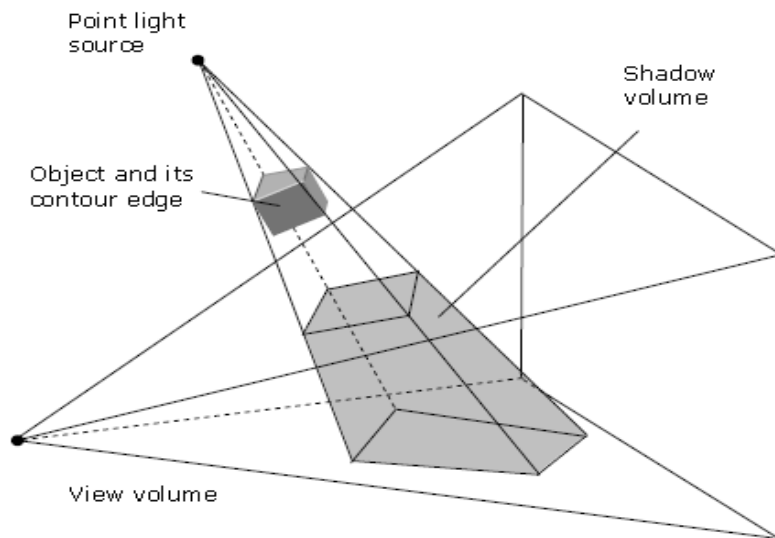
$$y_s = (y_l z_p - y_p z_l) / (z_p - z_l)$$

Shadow scanline – functioneaza doar cu surse de lumina punctiforme si necesita o faza de preprocesare: creaza o structura de perechi de poligoane care se pot umbri; perechile se determina prin proiectia poligoanelor pe o sfera cu centrul in sursa de lumina.



Algorithm: 1. Se creaza structura de date secundara; 2. Se determina daca exista umbre proiectate pe poligonul care este scanat de linia de baleaj; 3. Daca nu exista algoritmul continua in mod normal; 4. Daca exista umbre: folosind sursa de lumina ca centru al proiectiei, se genereaza o umbra pe planul poligonului curent; 5. Pt fiecare pixel de pe linia de baleaj, se determina daca este in umbra sau nu: daca poligonul care arunca umbra nu acopera segmentul liniei de baleaj se traseaza fara umbra, daca poligonul acopera complet segmentul atunci culoarea pixelului depinde de nr de umbre proiectate pe el, daca poligonul acopera partial segmentul atunci acesta trebuie divizat si se aplica recursiv algoritmul.

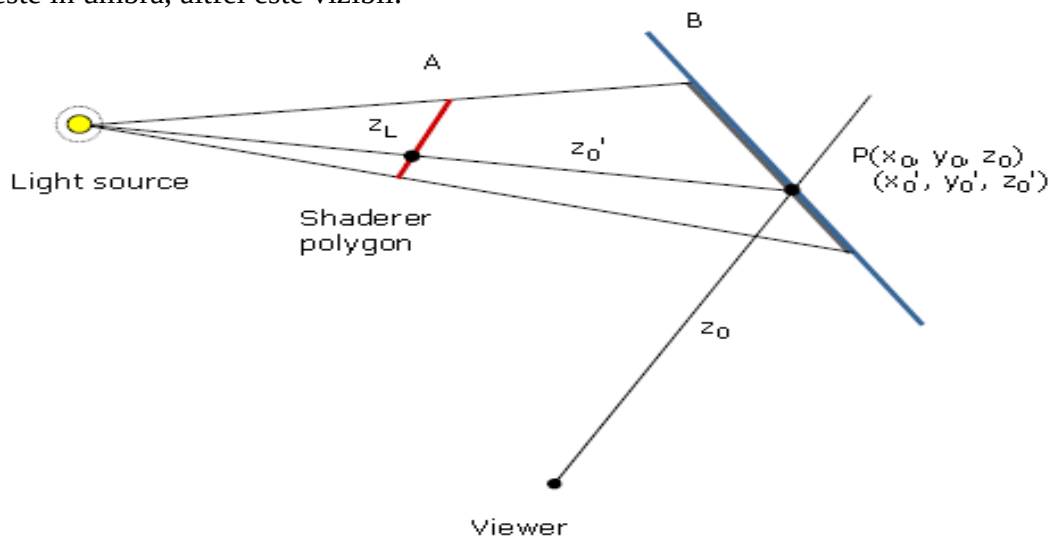
Umbre de volum – un volum invizibil de spatiu maturat de umbra unui obiect; volumul infinit e definit de drepte care pornesc de la sursa si trec prin varfurile obiectului; este finit daca se intersecteaza cu volumul de vizualizare; toate fetele obiectelor care se afla in acest volum au umbre proiectate pe ele.



Umbre din transformari – Se bazeaza pe algoritmi Weiler-Atherton pt decuparea suprafetelor ascunse si Weiler pt decuparea poligoanelor. Folosind pozitia sursei de lumina se identifica poligoanele ascunse. Acestea sunt umbrite, iar restul poligoanelor sunt luminate. Daca se modifica pozitia obs se observa fetele umbrite.

Z-buffer pt umbre. Este cea mai simpla abordare de calcul a umbrelor. Avem nevoie de un buffer Z pt fiecare sursa de lumina. Doi pasi ai algoritmului:

1. Scena este trasata si informatii despre adancime sunt stocate in z-buffer folosind sursa de lumina ca pct de vizualizare. Nu sunt calculate intensitati. Se determina poligoanele vizibile de sursa de lumina.
2. Trasam scena folosind algoritmul Z-buffer: Daca pct $P(x, y, z)$ este vizibil in sistemul centrat pe obs, calculeaza pct $P(x_0, y_0, z_0)$ ca fiind acelasi punct dar in sistemul centrat pe sursa de lumina. Compara cu valoarea din Z-buffer pt pct P: daca in buffer la pozitia (x_0, y_0) este o valoare z' mai mica decat z_0 atunci pct este in umbra, altfel este vizibil.



Putem folosi acelasi Z-buffer pt diferite pct de vizualizare;insa avem nevoie de un Z-buffer pt fiecare sursa de lumina; daca lucram in virgula flotanta un poligon s-ar putea umbri pe el; sa se evite calcularea Z-bufferului de umbra in faza de precalculare si sa se calculeze adancimea doar a obiectelor vizibile.

TEXTURI – Sunt modele grafice reprezentate sub forma unor matrici de pixeli. Maparea – transpunerea acestei matrici pe suprafața unui obiect.

Atribute ale texturii: culoarea suprafeței (coeficientii reflectiei difuze), reflexia speculară și difuză este legată de maparea mediului înconjurător, perturbarea vectorului normală (modificarea nivelului de reflexie), componenta speculară legată de asprimea suprafeței, transparentă (gen. ob. complete, nor). Exceptând maparea mediului, toate texturile se conectează la obiect, devenind o parte din el și se modifică o dată cu el.

Materiale. Texturi legate – rămân atașate după mapare și nu se modifică. Texturi care depind de observator – depind de unghiul de vizualizare (ex. crom, reflexia și refractia). Maparea umflăturilor – efectul de suprafață deformată creat de modificarea vectorilor normali; calculând în lum. prin modelul de reflectie local se obțin efecte texturale.

Maparea texturilor 2D – o textură 2D, îi se asociază fiecărui pct. pe obiect și se proiectează pe ecran. Se pot folosi doar obiecte poligonale, este dificil în cazul obiectelor discontinue, este ușor pt. suprafețe parametrice cubice. Prin antialiasing se pierd informații: nu pot asocia fiecărui punct de pe obiect un pixel.

Maparea inversă – Se ia un pixel și determin suprafața din textură care se va mapa pe acel pixel. Pași: 1. Considerăm 4 pixeli conectați; 2. Transpun obiectul în spațiul ecran; 3. Realizez parametrizarea suprafeței. Depinde pe ce zonă a texturii pică pixelul. Parametrizarea suprafeței se realizează printr-o funcție care îi atribuie fiecărui punct de pe poligon un punct de pe textură. Se ia un pct. de pe obiect, se trage o linie prin textură, și culoare prin care trece se înscrie într-o arie formată din cei 4 pixeli, a) intensitatea fiind o funcție a factorului de umplere sau b) cul. pixelului este aceeași cu a pixelului din centrul mapării.

Parametrizarea suprafețelor - se asociază un punct de pe obiect $X(x,y,z)$ cu un punct de pe textură (u,v) . Maparea poligoanelor: pb. locală, maparea fiecărui punct interior al poligonului într-un pct. de pe textură; globală, pentru obiecte maparea fiecărui poligon dintr-un nr. de poligoane conectate. Cunoșcând maparea vârfurilor obiectului pe textură putem afla maparea fiecărui punct în poligon pt.: renderizare, tăiere, scalare, deformare. Dacă nu se cunoaște fct. de mapare pt. fiecare pct. trebuie determinată funcția analitică corespundătoare: liniară într-o direcție sau non-liniară care divide poligonul inițial într-un set de zone liniare.

Maparea în 2 faze – Unifică maparea pe texturi cu environment mapping. Definim o suprafață înconjurătoare, mapăm textura pe ea și apoi pe obiect. Prima fază - mapare S: mapează textură 2D pe o suprafață simplă intermediară 3D. A doua fază – mapare O: mapează textură 3D pe suprafața obiectului. Pt. S mapping suprafețe intermediare: plane, suprafața laterală a unui cilindru, fața unui cub, suprafața unei sfere. Pt. O mapping intersecție dintre obiect cu suprafața intermediară poate fi: prin rază reflectată, prin normala obiectului, printr-o linie ce porneste dintr-un pct. interior, prin normala suprafeței intermediare.

Maparea inversă în 2 faze – Parametrizez cei patru pixeli (colturi) a.i. să corespundă cu 4 puncte de pe obiect, găsesc corespondentul fiecărui pct. pe suprafața intermediară și determin poziția corespunzătoare din textură pt. acest pct.

Mapare prin modelare – Folositoare pt. obiecte care pot fi obținute prin deformarea unui cilindru, sfere sau alte forme geometrice. 1. Deformăm obiectul sub forma unui cilindru; 2. Mapăm textura pe cilindru; 3. Deformăm obiectul la forma originală.

Mapare pe suprafețe parametrice bicubice 1. Divizăm suprafața în spațiul obiect; 2. În același timp realizăm o divizare proporțională în spațiul texturii; 3. Suprafața se divide până când acoperă cel mult un pixel; 4. Valoarea necesară din textură se obține din subdiviziunea curentă de la nivelul texturii. O altă abordare ar fi ca suprafețele obiectului să fie subdivizate în micro poligoane și „flat shaded” (umbre dif. pe fiecare poligon) cu valori din subdiviziunea corespundătoare în textură.

TEXTURI 3D – sunt obtinute prin generare procedurala.

Hipertexturile – obiectele nu au suprafata; ea este definita de o functie de densitate, iar textura este aplicata in intreg spatiul obiect.

Mapare cromata – 1. Se calculeaza vectorul normala pt pct P al obiectului;

2. Se determina vectorul de reflexie speculara in functie de N si de pozitia obs;

3. Se determina coordonatele u si v din domeniul texturii.

Maparea refractiei – suprafata intermediara este un plan prin care trece raza refractata.

Maparea mediului – raza reflectata intersecteaza suprafata intermediara intr-un punct, acel punct de pe textura se mapeaza pe punctul de pe obiect. Pe fiecare fata a unui cub se mapeaza mediul observat de pe fata respectiva.

Maparea umflaturilor – 1. Modificari unghiulare ale vectorului normala;

2. Se aplica modelul de reflexie locala pt a calcula intensitatea luminii.

Modificarile vectorilor $\mathbf{t_b}$ sa fie independente de orientarea si pozitia obiectului. Deformarea trebuie realizata in coord locale relativ la suprafata obiectului. Fie $\mathbf{O}$ un pct de pe suprafata obiectului.

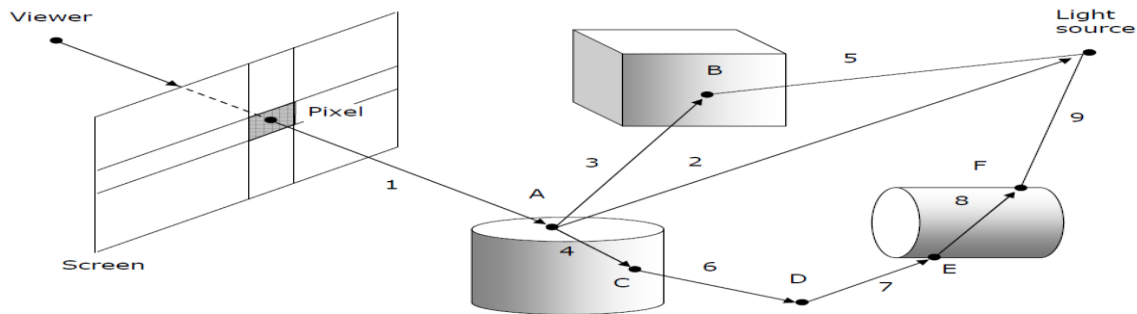
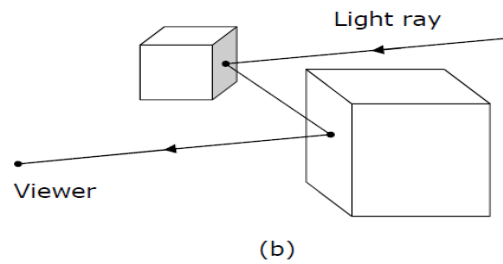
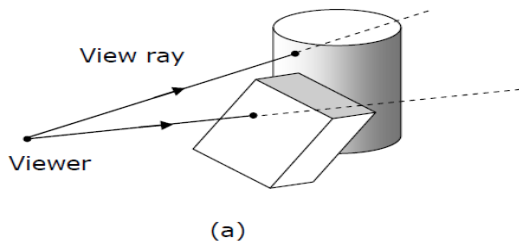
Vect norm: $\mathbf{N} = \mathbf{O_u} \times \mathbf{O_v}$. Functia de deformare este definita in functie de 2 vectori $\mathbf{A} = \mathbf{N} \times \mathbf{O_v}$ si $\mathbf{B} = \mathbf{N} \times \mathbf{O_u}$. Functia deformeaza vectorul $\mathbf{N}$ in $\mathbf{N'}$ ($\mathbf{D} = \mathbf{B_uA} - \mathbf{B_vB}$; $\mathbf{N'} = \mathbf{N} + \mathbf{D}$; $\mathbf{B_u}$ si $\mathbf{B_v}$ sunt derivatele partiale ale functiei de perturbatie $\mathbf{B(u,v)}$). Maparea depinde de factorul de scalare al obiectului; ex: daca factorul e 2, $\mathbf{N}$ creste de 4 ori, dar lungimea lui $\mathbf{D}$ creste de 2 ori. Astfel functia de perturbatie nu e scalata, iar umflaturile sunt netede.

RAY TRACING

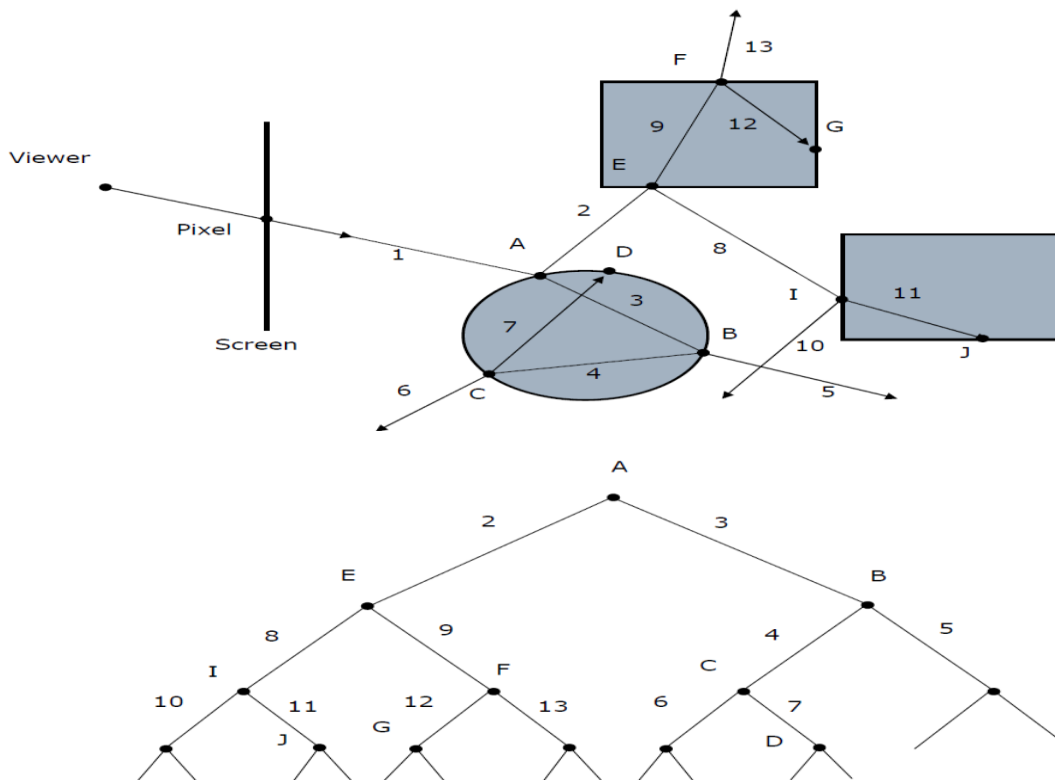
Algoritmul de baza

Functioneaza complet cu obiecte aflate in spatiu

Culoarea si intensitatea la un punct sunt obtinute prin trasarea unei raze de la ochi prin punctul de pe scena. Iluminarea este o contributie a iluminarii directe, reflectata si transmisa de raze.



Recursiv:



Calcularea intersecțiilor – mari consumatoare de timp (95% din timpul de prelucrare)

Accelerarea calcului intersecțiilor se face: Eliminând volume, evitând calculul intersecțiilor din exteriorul cutiilor

Intersection: ray-sphere:

Vizualizare raze: $P1(x_1, y_1, z_1)$ și $P2(x_2, y_2, z_2)$

Ecuatiile parametrice ale razei de vizualizare:

$$x = x_1 + (x_2 - x_1)t = x_1 + it$$

$$y = y_1 + (y_2 - y_1)t = y_1 + jt$$

$$z = z_1 + (z_2 - z_1)t = z_1 + kt$$

O sfera cu central (l, m, n) și raza r : $(x - l)^2 + (y - m)^2 + (z - n)^2 = r^2$

Inlocuind pentru x, y și z : $at^2 + bt + c = 0$

unde:

$$a = i^2 + j^2 + k^2$$

$$b = 2i(x_1 - l) + 2j(y_1 - m) + 2k(z_1 - n)$$

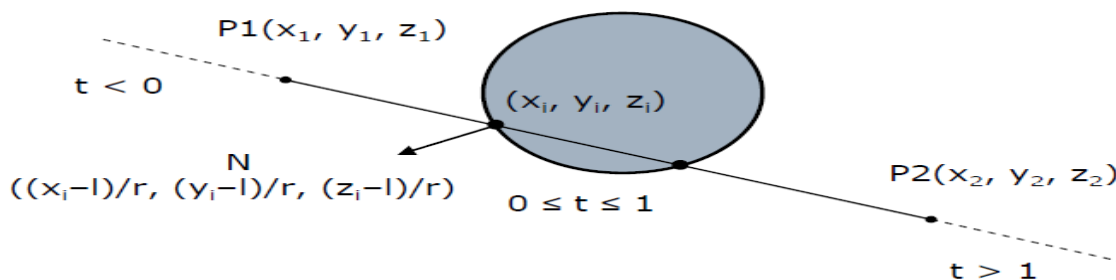
$$c = l^2 + m^2 + n^2 + x_1^2 + y_1^2 + z_1^2 + 2(-l x_1 - m y_1 - n z_1) - r^2$$

Dacă $k < 0$, linia nu intersectează sfera

$K=0$, linia este tangenta la sfera

$K > 0$, linia intersectează sfera în 2 puncte

Valoarea lui t da poziția intersecției în raport cu $P1$ și $P2$.



Accelerarea deciziei cu privire la intersecție:

1. Găsiți dacă originea razelor se află în afara sferei
2. Aflați cel mai apropiat punct al razei de centrul sferei
3. Dacă raza e în exterior și îndreptată înafara sferei, raza nu trebuie să se intersecteze cu sfera
4. Altfel, găsiți distanța pătrată de la cea mai apropiat punct al razei de suprafața sferei
5. În cazul în care valoarea este negativă, cu raza nu intersectează sfera
6. Altfel, din cele de mai sus, găsiți distanța dintre raza și suprafața
7. Se calculează coordonatele intersecției
8. Se calculează normala la punctul de intersecție

Intersection: ray-convex polyhedron:

Se testează raza individual pentru fiecare poligon

Determinate ecuația planului poligonului

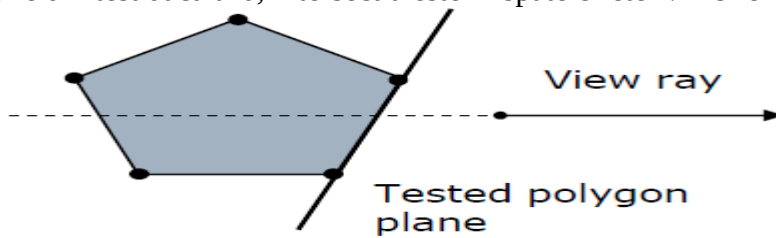
Se verifică intersecția dintre plan și raza

Se verifică dacă intersecția este în interiorul poligonului

Dacă ecuația planului poligonului este: $ax+by+cz+d=0$

atunci intersecția cu linia este dată de $t = -(ax_1+by_1+cz_1)/(ai+bj+ck)$

Iesire din test daca $t < 0$, intersectia este in spatele fetei vizibile

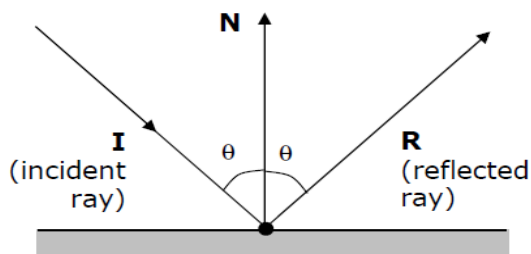


Reflectie + refractie

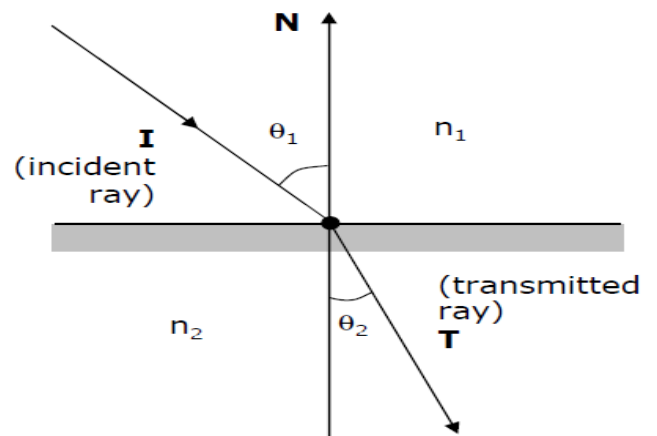
Legea lui Snell: $\frac{\sin \theta_1}{\sin \theta_2} = \frac{n_2}{n_1}$ $n = \frac{n_2}{n_1}$

In 1984 Heckbert si Hanrahan determina formula pentru raza rfractata:

$$\cos \theta_2 = n \left[1 - \frac{1}{n^2} (1 - \sin^2 \theta_1) \right]^{\frac{1}{2}}$$



(a) $R = I + 2N \cos \theta$



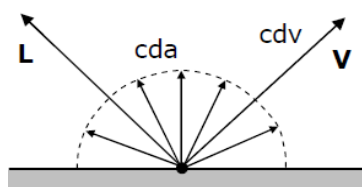
$$(b) \quad T = \frac{1}{n} I - (\cos \theta_2 - \frac{1}{n} \cos \theta_1) N$$

Illumination and reflection model

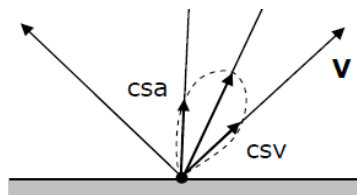
Iluminare globala – raze directe, reflectate si transmise

Iluminarea directa descrisa de modelul Phong:

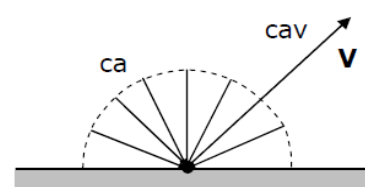
$I_{\text{Phong}} = \text{iluminare ambientala} + \text{iluminare difuza} + \text{iluminare speculara}$



(a) cdv – diffuse component in direction V ; cda – diffuse components in other directions.



(b) csv – specular component in direction V ; csa – specular components in other directions.



(c) cav – ambient component in direction V ; ca – ambient components from all directions

Iluminarea locala si globala (Radiosity)

Comparatie intre iluminarea locala si globala:

- Iluminarea locala: **Modelul Phong** (implementat in OpenGL)
- Iluminarea directa a suprafetei vazuta de catre observator
- Nu calculeaza umbre si reflexii reciproce intre obiecte
- Destul de rapida pentru grafica interactiva

Iluminarea globala: **Ray tracing**

- Reflexii speculare si refractii multiple
- Numai un singur pas al reflexiei difuze

Spatiul imagine: **Ray tracing**

Trasare inapoi dinspre observator

Calculare dependente de vizualizare

- Rezultatul: imagine raster (pixel cu pixel)

Iluminarea globala: **Radiatie** (Radiosity)

- Toate reflexiile difuze reciproce; umbre
- Metoda avansata: combinata cu reflexia speculara

Spatiul obiect: **Radiosity**

- Considera numai interactiunile difuzie-difuzie
- Calculare independente de vizualizare
- Rezultatul: model 3D, culoare pentru fiecare suprafata elementara (engl. surface patch)
- Pentru trasare poate fi utilizat OpenGL

Metoda Radiosity clasica

- Imparte suprafata in suprafete elementare (elemente).
- Modeleaza transferul luminii intre suprafetele elementare ca un sistem de ecuatii liniare.
- Consideratii importante:
 - Reflexia si emisia sunt difuze
 - Reflexia difuza este egala in toate directiile
 - Radianta este independenta de directie
 - Nu depinde de mediu (obs: nu exista ceata)
 - Nu există refractie (numai suprafete opace)
 - Radiatia este constanta pentru un element (suprafata elementara)
 - Se calculeaza separat pentru R, G, B

Masurarea iluminarii

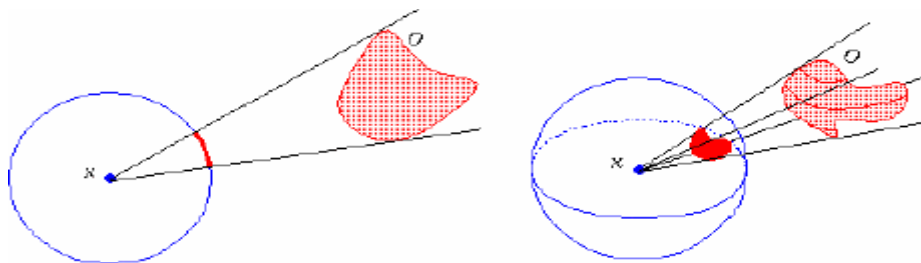
Unghiul solid

Unghiul 2D se defineste ca unghiul subintins de catre obiectul O din punctul x:

- Lungimea proiectiei obiectului pe cercul unitar centrat in x
- Masurat in radiani ($0 \dots 2\pi$)

Unghiul solid 3D este unghiul subintins de obiectul O din pozitia x:

- Aria proiectiei obiectului pe suprafata sferei unitare centrate in x
- Masurat in steradiani ($0 \dots 4\pi$)



Puterea radiantă și radiația

Puterea radiantă **P**:

- Rata cu care este transmisă energia luminoasă
- Puterea = energie / timp

Densitatea fluxului Φ :

- Puterea radiantă pe unitatea de suprafață
- Fluxul = puterea / suprafață

Iradieră **E**: densitatea de flux incident la suprafață

Radiația **B**: densitatea de flux care iese din suprafață. Radiația = puterea / suprafață.

Densitatea de flux într-un punct $\rho(x) = dP / dx$

Structura de calcul a radiației

- Împarte scena în suprafețe elementare
- Calculează factorii forma dintre elemente
- Independent de iluminare
- Rezolvă ecuația liniară a radiației
- Independent de vizualizare
- Trasează folosind hardware standard

Puterea într-un punct după o anumită direcție

Intensitatea radiantă **I**:

- Puterea radiată pe unitatea de unghi solid de către o sursă punctiformă
- Intensitatea radiantă = puterea / unghiul solid

Intensitatea radiantă în direcția ω :

$$I(\omega) = dP / d(\omega)$$

Radianța **L(x, ω)**:

- Densitatea de flux în punctul x spre direcția ω
- Radianța = puterea / (suprafață * unghiul solid)

Factorii forma

Factorul forma **F_{ij}**: este raportul dintre lumina care pleacă de pe elementul i și ajunge pe elementul j .

Factorul forma depinde de: Forma elementelor i și j , Orientarea relativă a celor două elemente, Distanța dintre elemente, Ascunderea de către alte elemente

Ecuația factorului forma

Definim:

- Unghiurile polare θ și θ' dintre normale și raza dintre x și y .
- Funcția de vizibilitate este $v(x, y) = 0$, dacă raza de la x la y este întreruptă, altfel $v(x, y) = 1$.
- Distanța dintre x și y este r .

Calcularea factorilor forma

Două metode principale de calcul:

- Hemicube: utilizează modulele hardware pentru z-buffer
- Ray casting (poate fi lentă)

Ambele metode generează efect aliasing (contururi zimțate, non netede).

ANIMATIA

Animatia este afisarea rapida a unei secvente de imagini putin diferite, creind iluzia miscarii.

Animatia prin modificarea caracteristicilor

Animatia realizata prin modificarea unor caracteristici ale sistemului de vizualizare.

1. Pentru obiectele din scena: Pozitia, Orientarea, Dimensiunea, Forma, Culoarea, Transparenta
2. Pentru camera de luat vederi: Punctul de vedere, Punctul de interes, Unghiul de vedere
3. Pentru sursa de lumina: Pozitia, Intensitatea luminii

Animatia clasica bazata pe celule 2D

Tehnica traditionala de animatie este foarte laborioasa:

1. Incepe prin definirea unui scenariu
2. Artistul principal deseneaza cadrele cheie care definesc actiunea
3. Alti artisti deseneaza cadrele intermediare pentru a completa animatia

Animatia automata prin cadre cheie

1. Fragmentarea automata a cadrelor cheie

Inlocuieste desenarea manuala a cadrelor intermediare prin folosirea unor algoritmi de interpolare. Cadrele cheie corespund valorilor parametrilor in anumite momente de timp.

Avantajul este ca prin calculator se poate repeta prelucrarea cadrelor.

Dezavantajul consta in calitatea fragmentarii mai slaba decat cea obtinuta manual. Pentru a obtine rezultate bune trebuie realizate mai multe cadre intermediare.

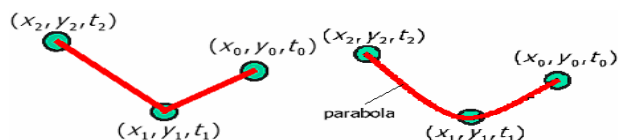
2. Interpolarea liniara

Este cea mai folosita tehnica de calculare a valorilor intermediare pentru un interval dat. Fiind dati doi parametri p_0 si p_1 la momentele t_0 si t_1 , o valoare intermediara este data de:

$$p(t) = \frac{t_1 - t}{t_1 - t_0} p_0 + \frac{t - t_0}{t_1 - t_0} p_1$$

Avantaje: rata schimbarii de-a lungul unui segment este constanta si poate fi controlata.

Dezavantaj: exista discontinuitati in toate pozitiile intermediare.



3. Controlul vitezei

Fiind data o anumita viteza de modificare (animatie), se defineste viteza variabila de modificare. Fiind data o functie $_ = f(t)$ se trece la variabila $_$ in expresia de interpolare liniara:

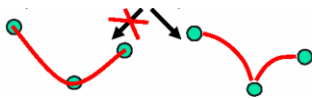
$$p(\tau) = p(f(t)) = \frac{f(t_1) - f(t)}{f(t_1) - f(t_0)} p_0 + \frac{f(t) - f(t_0)}{f(t_1) - f(t_0)} p_1$$

4. Interpolarea Spline

Alaturi de interpolarea liniara se foloseste interpolarea curbilinie pe baza curbelor Spline.

Avantaje: Permite un control continuu al modificarii. Numarul de cadre intermediare poate fi determinat in functie de calitatea ceruta pentru animatie

Dezavantaje: Calculul punctelor de control al curbelor spline trebuie sa se bazeze pe constrangeri de interpolare din punctele de control. Sunt necesare informatii suplimentare cum ar fi vectorii tangenta. Calcularea curbelor splines necesita resurse mai multe (ex. timp). Calcularea curbelor splines este mai greu de implementat.



Tipuri de curbe splines:

B-splines: permit control automat al continuitatii, insa numai aproximeaza punctele de control.

Non-uniform B-splines: necesare pentru a intrerupe continuitatea.

Non-uniform rational B-splines (NURBS): permit controlul local suplimentar al formei curbei.

Bézier: necesita informatii suplimentare pentru fiecare segment si nu mentin automat continuitatea. Interpoleaza insa, capetele segmentelor.

Hermite: necesita vectorii tangenta pentru fiecare cadru cheie.

Catmull-Rom: asigura continuitatea primei derivate (traectorie neteda)

Beta-splines: permite continuitatea automata, controlul asupra tangentei si aproximeaza lungimea arcului fiecarui segment.

5.Traectoria animatiei

Animarea pe o traectore consta din calcularea pozitiilor intermediare. Pentru deplasarea pe traectorie sunt necesare doua facilitati de baza:

1. Controlul independent al vitezei de-a lungul traectoriei
2. Controlul continuitatii

Curbele spline permit controlul continuitatii, insa controlul vitezei este mai dificil. Interpolarea pozitiei pe curba spline permite controlul continuitatii pozitiilor, dar:

- Un increment egal al parametrului spline nu corespunde intotdeauna unui increment egal in distanta de-a lungul curbei.
- Segmente diferite ale curbei spline corespunzatoare aceleiasi lungimi parametrice pot avea lungimi fizice diferite.
- Daca curba spline este parametrizata direct cu variabila timp, obiectele se vor misca cu viteza ne-uniforma.

6.Parametrizarea lungimii arcului traectorie

Se parcurg urmatoorii pasi de transformare:

1. Fiind dat un pas spline $P(u) = |x(u), y(u), z(u)|$, se calculeaza lungimea arcului spline ca o functie de u : $s = A(u)$.
2. Se determina functia inversa $A(u)$: $u = A^{-1}(s)$.
3. Se substituie $u = A^{-1}(s)$ din $P(u)$ pentru a afla parametrizarea caii de miscare prin lungimea arcului, s : $P(s) = P(A^{-1}(s))$.

Observatie: parametrul u (si astfel s) trebuie sa fie parametru global pentru toate segmentele curbei spline initiale.

7.Controlul vitezei

Controlul vitezei de-a lungul traectoriei spline:

- Fie $s = f(t)$ definitia distantei pe curba spline, ca o functie de timpul t .
 - Functia $f(t)$, ca o valoare scalara, poate fi prelucrta printr-o tehnica de animatie functionala (ex. o alta curba spline).
 - Functia $f(t)$ poate fi exprmata ca o integrala a functiei viteza, $v(t) = df(t)/dt$.
- Observatie: integrala unei functii spline poate fi aflata analitic, prin intermediul punctelor de control.
- Traectoria de miscare ca o functie de timp este data de: $P(t) = P(f(t)) = P(A^{-1}(f(t)))$.

Interpolarea orientarii

Interpolarea orientarii se calculeaza indirect prin interpolarea unghiurilor si nu prin matrici de transformare:

- In 2D orientarea e definita de un singur unghi, iar animarea este relativ simpla
- In 3D se utilizeaza trei unghiuri, iar topologia este a unei sfere in patru dimensiuni. Interpolarea este mai greu de vizualizat.

Metode pentru interpolarea orientarii:

Unghiuri Euler. Se folosesc trei unghiuri: rotatie dupa x , apoi rotatie dupa y si apoi rotatie dupa z .

Dezavantaje: dificil de rezolvat problema inversa – fiind data orientarea, care sunt unghiurile?

Avantaje: Larg folosita in practica, usor de implementat, metoda necostisitoare.

Quaterni. Patru elemente analoage numerelor complexe.

Dezavantaje: necesita matematica mai multa, interpolare costisitoare.

Avantaje: rezolvare simpla a problemei inverse, interfata utilizator simpla.

Animatie prin miscarea camerei de luat vederi

Animatia camerei poate fi realizata prin combinarea interpolarii pozitiiilor si a orientarii. Exista totusi citeva cerinte specifice cum ar fi:

- Camera trebuie sa fie intotdeauna orizontala, daca nu se specifica alta directie.
- Imaginile obiectelor de interes trebuie sa fie stabile in planul filmului.
- Specificarea miscarii camerei trebuie sa respecte anumite conditii cinematice.

Unelte pentru animatia formei

Fiind dat un model cu un foarte mare numar de virfuri, acestea pot fi grupate si prelucrate ca o unitate. Exemplu, punctele care descriu antebratul se misca mai mult sau mai putin ca un corp rigid. Conectivitatea intre grupuri poate fi reprezentata prin legarea fiecarui grup la un „os” care face parte dintr-un „schelet” articulat. Oasele si virfurile asociate sunt tratate ca un singur obiect rigid in operatiile de transformare. Miscarea oaselor este constrinsa de legaturile din schelet. Diferitele forme de conexiune (rotatie, unghi, sfera) permit diferite forme de miscare. Animatia poate fi vizualizata rapid folosind trasarea doar a scheletului. In final modelul este vizualizat fara oase.

Animatie prin deformare libera

Uneori, scheletul nu poate reda modificarea formei unui obiect. De exemplu, turtirea si alungirea unei mingi care sare pe pamint. Astfel de modificari ale formei pot fi exprimate folosind deformarea.

Deformarea:

- Modifica spatiul dintre obiecte.
- Este o transformare neliniara.
- Noile coordonate ale fiecarui punct din spatiu sunt determinate ca functii ale vechilor coordonate

Metoda:

1. „Extensia deformabila” este plasata in jurul fiecarui parti a modelului ce urmeaza a fi animat.
 2. Coordonatele tuturor punctelor din interiorul extensiei sunt determinate relativ la cadrul dat de extensie. Consideram ca un virf P poate fi exprimat ca $[u, v, w]$ relativ la coordonatele extensiei.
 3. Noile coordonate ale lui P sunt date de un tensor produs spline Bezier $B_{n1, n2, n3}(u, v, w)$ avind punctele de control P_i, j, k
 4. Daca P_i, j, k are coordonatele: $P_i, j, k = [i/(n1+1), j/(n2+1), k/(n3+1)]$ atunci transformarea este data de identitatea: $[u, v, w] = B_{n1, n2, n3}(u, v, w)$. Apoi, punctele de control sunt fixate la aceste valori si sunt deplasate pentru a da efectul de deformare.
 5. Conditile de continuitate pot fi impuse.
- De exemplu: mina este deformata dar nu si bratul. Punctele de control de-a lungul marginii extensiei care taie incheietura miinii nu trebuie deplasate si nici urmatorul nivel. Aceasta mentine continuitatea ambelor pozitii si derivate de o parte si de alta a incheieturii miinii.
6. In animatia reala se fac deformari mici.

Deformarea bazata pe schelet si deformarea libera pot fi folosite simultan. Numarul punctelor de control in deformarea libera poate fi foarte mare. Daca se face miscarea in grup relativa la schelet, se poate obtine un control foarte bun al animatiei.

Cinematica directa si cinematica inversa

Cinematica se defineste ca studiul miscarii independenta de fortele care determina miscarea. Considera pozitia, viteza si acceleratia.

Cinematica directa (Forward Kinematics): determinarea Pozitiilor, Vitezelor, Acceleratiilor tuturor legaturilor dintr-un model articulata, fiind date Pozitia, Viteza, Acceleratia nodului radacina al modelului si toate transformarile dintre legaturi.

Cinematica directa este importanta in animatia bazata pe cadre scheletizate. Este o metoda relativ usor de implementat.

Cinematica inversa (Inverse Kinematics) se definește ca determinarea mișcării legăturilor intermediare dintr-un corp articulat fiind dată mișcarea unor legături cheie.

Caracteristici ale metodei bazate pe cinematica inversă:

- Neliniară, nedeterminată.
- Complexitatea determinării soluției este proporțională cu numărul legăturilor libere.
- Îmbinare liberă dintre două fixe poate fi rezolvată eficient numai prin sacrificarea unui grad de libertate.
- Sunt necesare constrângeri suplimentare pentru a obține o soluție unică și stabilă. Exemplu, considerarea forței gravitaționale, constrângerile impuse de unele tipuri de articulații (rotatie vs sferică, genunchi vs umăr).
- Obiective suplimentare de optimizare.

Rezultând problema de optimizare rezolvată iterativ ca un proces de animație.

Exemplu: minimizarea energiei cinetice a structurii.

Tehnici alternative de animație: Transformare morfologică (Morfing), Captarea mișcării, Sisteme de particule, Grup (Flocking)

1. Transformarea morfologică

Este transformarea de interpolare între imagini. Efecte bune la un cost redus. Pași principali: 1. Partitionarea imaginilor în piese corespunzătoare. Exemplu, ochi cu ochi, etc. 2. Interpolarea pieselor (geometria) 3. Interpolarea atributelor (pixeli, culori) 4. Filtrare

2. Captarea mișcării

Animațiile complexe necesită o multitudine de cadre cheie (exemplu, mersul unui umanoid). Captarea pozițiilor cheie de la obiecte reale. Sunt înregistrate în model pozițiile unor puncte cheie de pe obiectul real (ex. un actor).

Tehnologii de urmărire a poziției:

1. Senzori electromagnetici pentru poziție și orientare. Necesită cabluri la fiecare senzor. Citirea poate fi perturbată de cimpuri magnetice. Spațiul de mișcare este limitat.
2. Triangulație prin ultrasunete - Simplu dar mai puțin precis decât senzorii magnetici.
3. Triangulație optică - Sunt atașați emițători la obiectul urmărit. Două camere triangulează poziția. Poate urmări mai multe puncte, fără a folosi cabluri.
4. Costume de corp - Senzori care determină unghiurile din încheieturi sunt inserați în țesătura hainelor.

3. Sisteme de particule

Cum se pot modela obiecte „fuzzy” (vagi) cum ar fi: Obiecte cu margini neclare, Obiecte cu margini care se schimbă, Obiecte cu comportament haotic

Exemple: nori, foc, apă, iarba, blana, etc.

Principiul: modelează obiectele fuzzy ca mulțimi de particule.

Fiecare particulă are proprietăți cum ar fi: Geometrie (punct, sferă, segment de dreaptă), Poziție, Vector viteză, Dimensiune, Culoare, Transparență, Stare, Durată de viață

Animația sistemelor de particule:

Noi particule se nasc, cele vechi mor.

- Informația de stare descrie tipul particulei
- La fiecare moment: Actualizează atributele tuturor particulelor, Șterge particulele vechi, Creează noi particule, Afinează starea curentă a tuturor particulelor
- Pentru antialiasing, desenează un segment de dreaptă din vechea spre noua poziție
- Pentru a desena iarba sau blana, desenează întreaga traiectorie a particulei

4. Animația grupurilor de animale

Animația grupurilor de animale: stoluri de păsări, pesti, turme de animale, etc.

Sunt prea mulți indivizi pentru a fi animați fiecare prin cadre cheie. Elemente mai puține decât în sistemele de particule, dar cu mai multe interacțiuni. Se impune evitarea ciocnirilor cu alte obiecte (ex. clădiri, copaci) sau între ele. Grupul are un centru: fiecare membru încearcă să rămână membru al grupului, centru grupului este unul global, nu permite divizarea grupului pentru a trece un obstacol.